Studying structural properties and exact models for solving the Job Sequencing and Tool Switching Problem

Shankar Narayanan

Internship report

Advisors:

Prof. Rafael Colares Prof. Annegret Wagler

Supervisor:

Dr. Renaud Chicoisne

Laboratoire d'Informatique, de Modélisation et d'Optimisation des Systèmes (LIMOS)

August 18, 2026

Abstract

The Job Sequencing and Tool Switching Problem (SSP) seeks an ordering of jobs on a single flexible machine, together with a tool-loading policy for a capacitated magazine, that minimises the number of tool switches. For a fixed order the loading is solved exactly by a classical greedy rule, so the difficulty lies in the ordering, and the problem is NP-hard. In practice the SSP is solved by a two-phase decomposition: group the jobs into as few magazine-fitting batches as possible, then sequence the batches. This report asks two questions about that standard method. It answers most of both. *How wrong can the decomposition be?* We prove that its error—the gap—is exactly the price of insisting on the fewest batches; that the gap vanishes in several broad classes yet can grow without bound; and we determine its worst-case behaviour quantitatively: an exact cost formula when the batch number is three, bounds that are tight at every known extremal instance, and a sharp separation between the heuristic's configuration-walk model—for which the natural candidate condition gap $\leq K^* - 2$ fails by an explicit certificate—and the heuristic as actually run, whose KTNS loading step beats the walk on the extremal instances and for which the condition is reopened. We also prove that any instance with optimum at most three has gap at most one. At $b = 3$ we prove the ratio bound $H/Z^* \leq 4/3$ whenever $K^* \leq 3$. A setup-cost analysis then shows the SSP collapses onto the grouping problem exactly when changeovers cost more than the gap—and real tooling data says they do. *Can the same grouping view solve the SSP exactly?* Here the obstacle is one number, the coverage bound $|U| - b$: we prove that every natural configuration relaxation is stuck at it, and a first benchmark of five exact methods on 1,421 standard instances—preliminary, with the definitive campaign still running—already indicates the suites split roughly in half between instances where this bound equals the optimum (easy for every method) and instances where no current method has traction. The numbers below are therefore reported as provisional evidence for that dichotomy rather than as final measurements. Every proved statement appears as a theorem in the text; exactly one conjecture remains, and it is labelled as such.

Contents

1 Introduction

Motivation. A flexible manufacturing cell built around a single computer-numerically-controlled machine can produce a wide variety of parts because it draws on a large library of cutting tools. The machine, however, holds only a limited number of tools at once, in a magazine of capacity b; producing the full part mix therefore requires tools to be exchanged between jobs. Each exchange interrupts production while the magazine is reconfigured, so tool switches are the dominant controllable cost in such cells [15]. When the set of tools needed by each job is fixed, two decisions remain: in what *order* to process the jobs, and which tools to *keep or evict* at each step. The Job Sequencing and Tool Switching Problem (SSP) is the joint optimisation of these two decisions so as to minimise the total number of switches. The same abstract structure—a capacity-limited store that serves set-valued requests which may be reordered—recurs well beyond machining, and the methods developed here travel with it. In printed-circuit-board assembly the "tools" are component feeders on a placement machine and the "jobs" are the boards; in high-volume colour printing they are ink cartridges on a press—the setting in which the SSP appears verbatim and is solved industrially by the very group-then-sequence decomposition this report analyses [7]; in warehouse order picking they are the items a picker can carry at once. In each case the scarce, reconfigurable resource and the freedom to reorder the demands are the whole of the problem, so nothing below is specific to cutting tools.

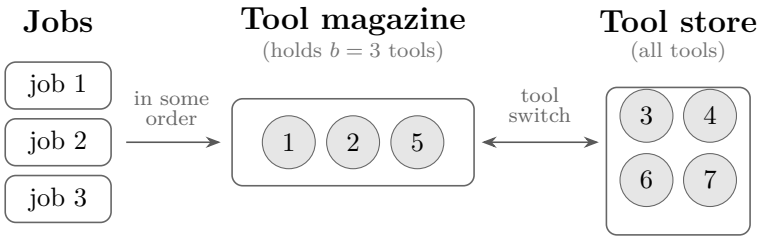

Figure 1: The physical setting. A computer-controlled machine draws tools from a magazine that holds only b at a time; the jobs are run in some order, and whenever a job needs a tool not currently loaded, a *tool switch* brings it from the store and evicts another—halting the spindle. Minimising the number of such switches over all job orders is the problem this report studies; Figure 2 strips it to its essentials.

Why it is worth solving. Two things make this more than a manufacturing curiosity. The first is that the cost is real and concentrated: a tool change halts the spindle—seconds to minutes of idle machine each time—and a cell that runs thousands of parts a day can lose hours of capacity to switches that a better order would have avoided. Saving switches is saving machine time, and machine time is money. The second reason is less obvious, and is what makes the problem live *research* rather than engineering. The way the SSP is solved in practice rests on a piece of folklore that has never been checked.

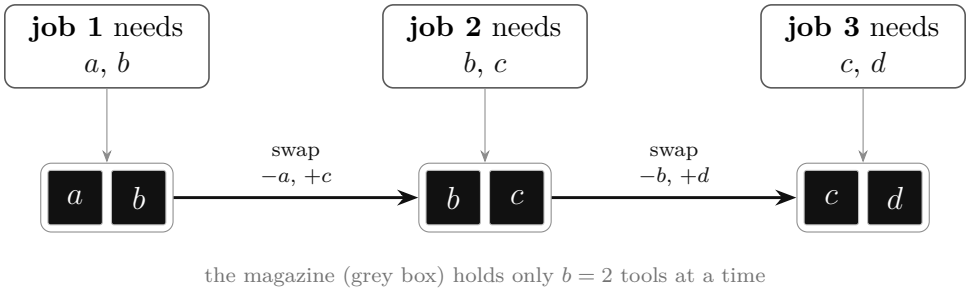

Figure 2: The problem in miniature. Three jobs run left to right on a machine whose magazine holds only $b = 2$ tools at once; a job can run only when all of its tools are loaded (upper row → lower row). Filling the magazine for job 1 costs two insertions; after that each job happens to share one tool with its predecessor, so just one tool is swapped in at each step—two further switches, four in total. Reorder the jobs, or enlarge the magazine, and that count changes. Choosing the order that minimises the total number of insertions, over all $n!$ orders, is the problem this report studies.

Practitioners, and the strongest published algorithms too, almost always *group* the jobs first and *order* the groups afterwards; this is fast, and on the instances people have tried it lands close to the optimum [7, 33]. Yet nobody has shown how far from optimal it can be, or characterised the instances on which it is exact: the survey literature records the heuristic's empirical success and its missing guarantee side by side [8]. The exact side has the same gap in reverse: the strongest compact relaxations of the SSP all reduce to a single elementary quantity—the number of tools used beyond what the magazine can hold—so a solver relying on them cannot prove optimality on the hard instances. This report addresses both gaps, and the same quantity, introduced in Section 2.5, governs both when the heuristic is exact and when the exact methods close.

Why the problem is subtle. The two decisions are coupled in opposite directions. For a *fixed* job order, the optimal loading policy is known in closed form—the Keep Tool Needed Soonest (KTNS) rule of Tang and Denardo [47], computable in $O(n|T|)$ time—so the loading subproblem is easy. The difficulty lies entirely in the ordering: a good order places jobs with similar tool requirements next to one another so that few tools change hands, but "similarity" here is a set-overlap relation with no underlying metric, and the capacity constraint couples positions that are far apart in the sequence. The joint problem is NP-hard [15, 29], and even the strongest compact integer formulations [9, 17, 29] become difficult beyond moderate sizes. This structure—an easy inner problem nested inside a hard ordering problem—is what decomposition methods are designed to exploit, and both lines of this work, the structural analysis and the exact methods, rest on it.

The grouping view, and the questions it forces. The SSP is closely tied to a simpler relative, the Job Grouping Problem (JGP), which asks only how few magazine configura-

tions suffice to serve every job. Grouping suggests both the standard heuristic—group the jobs, then order the groups—and a route to exact solution, because a *configuration* (a magazine-filling set of tools) is a natural unit to reason about: the SSP becomes a generalised travelling-salesman problem (GTSP) that visits, in some order, one configuration able to serve each job, paying for the tools that change between consecutive configurations [12]. Two questions follow, and the literature answers neither. First, how good is the heuristic? Group-then-sequence is the method of choice in applications—in colour printing, where the SSP appears verbatim, it comes close to optimal on industrial instances [7], and the strongest metaheuristics seed from JGP solutions [33]—but its record is entirely empirical: how far it can be from the optimum, and when it is exact, has not been analysed. Second, can the SSP be solved exactly through configurations? Branch-and-price works well for the JGP because its set-covering relaxation is tight [39]. Whether the configuration view gives an equally strong exact method for the SSP was open—and the answer turns on whether its natural relaxation can rise above the elementary bound $|U| - b$.

A single quantity links the two. These questions share an answer in that coverage bound $|U| - b$, the number of used tools in excess of the magazine capacity. It is at once the value every configuration relaxation attains and, as the gap analysis shows, the exact optimum on a large part of the instance space. Where it is tight, exact configuration methods close at the root and the heuristic tends to be exact; where it is loose, the heuristic's gap can open and the same methods branch heavily. That coincidence, developed in Sections 3 and 5, is the connecting thread of this report, and the configuration view that produces it is established once, in Section 2.

Contributions. The principal results are the following.

- *A reframing of the heuristic gap* (Section 3): we prove that the SSP optimum equals the minimum GTSP cost over *all* feasible groupings, not only minimum-cardinality ones. Consequently the two-phase heuristic's optimality gap is exactly the cost of restricting attention to minimum-cardinality groupings.
- *Worst-case guarantees* (Section 3.3): the matching lower bounds $Z^* \geq \max(K^* - 1, |U| - b)$; the ratio guarantee $H/Z^* \leq \min(b,\ K^* - 1,\ |U| - b)$; zero-gap classes valid for every b; and a family on which the gap grows linearly without bound.
- *The extremal case analysis* (Section 3.4): an exact formula for the heuristic cost at $K^* = 3$; quantitative gap bounds for every K^*, tight, for the walk model, at every known extremal instance; a certified counterexample eliminating the candidate gap $\leq K^* - 2$ for the walk model, together with the discovery that the KTNS loading step strictly beats the walk on those very instances—so the condition is reopened for the heuristic itself; the guarantee that $Z^* \leq 3$ forces gap at most one;

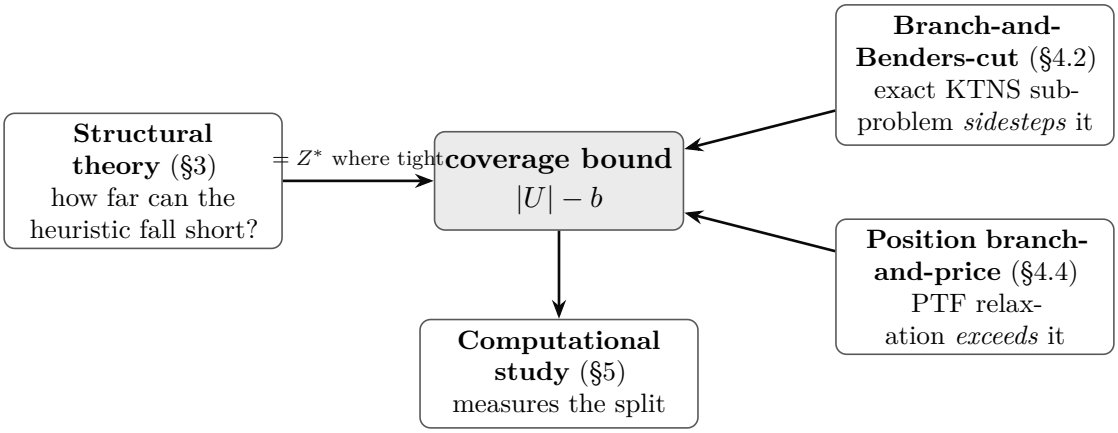

Figure 3: The shape of the argument. One quantity, the coverage bound $|U| - b$, is the hinge of the whole report: the structural theory shows it equals the optimum wherever it is tight (§3); the two exact methods are chosen precisely to get *past* it where it is loose—branch-and-Benders-cut by sidestepping the linear relaxation, position branch-and-price by strengthening it (§4); and the computational study measures how the instances split between the two regimes (§5).

and the 4/3 ratio bound at $b = 3$ proved for every $K^* \leq 3$, its $K^* \geq 4$ case being the one surviving conjecture.

- *Structure of the gap* (Section 3.5): the feasible groups form an independence system whose circuit clutter computes K^* as a hypergraph chromatic number, yet provably does not determine the gap—two instances with identical circuit structure and equal (K^*, Z^*) have different gaps—so any zero-gap criterion must read tool-level overlaps, as the exact $K^* = 3$ expression does. On the covering side, blocking duality yields the first non-idealness family (odd rings) for the set-covering relaxation behind grouping branch-and-price.
- *The setup-cost spectrum*: with a per-changeover setup cost ρ, the augmented problem *is* the SSP for $\rho < 1/(n-1)$ and collapses onto minimum-cardinality grouping—the heuristic exactly optimal—once ρ exceeds the gap; real tooling ratios sit far on the collapsed side.
- *Exact methods* (Section 4): a branch-and-Benders-cut solver whose subproblem is the polynomial KTNS oracle; and position-indexed formulations (the position–configuration formulation PCF, its symmetry-reduced variant PCF′, and the position–transition formulation PTF) with a fixed $O(n|T|)$ row set, their pricing subproblems, and an exact branch-and-price.
- *The computational instrument* (Section 5): a complete benchmark of all the methods above against the literature's compact models, quantifying the coverage bound's tightness on the standard suites (48.0% among solved instances) and demonstrating mechanically that configuration-based methods—Benders and branch-and-price alike—are bound-limited rather than implementation-limited.

Outline. Section 2 fixes notation, defines the SSP and KTNS, introduces the two-phase heuristic and the configuration–GTSP lens, proves the two structural lower bounds, fixes the switch-cost convention, and reviews the literature. Section 3 develops the structural theory of the heuristic gap; Section 4 presents the exact methods; Section 5 reports the experimental design and the competitiveness assessment; Section 6 concludes; and Section 7 sets out the work remaining in the project. Longer proofs are deferred to Appendix A. A reader pressed for time can follow the spine of the report through the heuristic's definition (Section 2.3), the exactness theorem and the extremal case analysis (statements in Sections 3.1 and 3.4), and the results table of Section 5.2.

2 Preliminaries

2.1 The Job Sequencing and Tool Switching Problem

We are given n jobs $J = \{1, \dots, n\}$, m tools $T = \{1, \dots, m\}$, a magazine capacity $b \in \mathbb{N}$ with $b < m$, and for each job j a set of required tools $T_j \subseteq T$ with $1 \le |T_j| \le b$. We write $U = \bigcup_{j \in J} T_j$ for the set of tools used by at least one job.

Definition 1 (Schedule and switch cost)**.** A *schedule* is a pair $(\pi, \mathbf{M})$ consisting of a permutation π of J and a sequence of *magazine states* $\mathbf{M} = (M_1, \dots, M_n)$, $M_i \subseteq T$, satisfying the capacity and feasibility constraints

$$|M_i| \le b \quad \text{and} \quad T_{\pi(i)} \subseteq M_i \qquad (i = 1, \dots, n).$$

With $M_0 = \varnothing$, its *switch cost* is the total number of tool insertions, $\sum_{i=1}^{n} |M_i \setminus M_{i-1}|$. The SSP asks for a schedule of minimum switch cost; this optimum is denoted Z^*.

Counting insertions is the standard cost model: under a fixed capacity each insertion into a full magazine forces exactly one removal, so insertions and removals differ only by boundary terms, which Section 2.6 makes precise.

Example 1 (A running instance: the 6-ring). We illustrate every definition on one instance, returned to throughout (Figure 4). The 6-*ring* has $n = m = 6$, capacity $b = 3$, and $T_j = \{j, j+1\}$ with indices taken modulo 6 (so $T_6 = \{6, 1\}$); the jobs form a cycle in which consecutive jobs share exactly one tool. Its job–tool incidence matrix (rows are

tools 1–6, columns jobs 1–6; a blank denotes 0) is

	1	2	3	4	5	6
t_1	1					1
t_2	1	1				
t_3		1	1			
t_4			1	1		
t_5				1	1	
t_6					1	1

Process the jobs in the cyclic order $1, 2, 3, 4, 5, 6$ and load by the rule of Section 2.2. The magazine evolves as

$$\{1,2,3\} \;\to\; \{1,2,3\} \;\to\; \{1,3,4\} \;\to\; \{1,4,5\} \;\to\; \{1,5,6\} \;\to\; \{1,5,6\},$$

inserting 3 tools to build the first magazine and then one tool at each of positions $3, 4, 5$: a switch cost of 6, equivalently 3 once the initial fill is not charged (Section 2.6). Section 3 shows that 6 is optimal, yet the two-phase heuristic of Section 2.3 returns 7. The 6-ring is thus the smallest instance on which grouping-then-sequencing is provably sub-optimal, and it recurs below as the canonical witness of a positive gap.

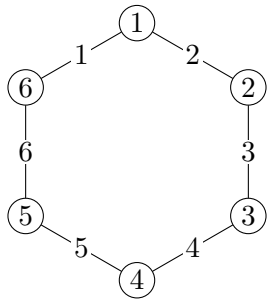

Figure 4: The 6-ring. Jobs 1–6 lie on a cycle; each edge is labelled by the single tool its two endpoints share ($T_j = \{j, j+1\}$). Adjacent jobs overlap in one tool; non-adjacent jobs are tool-disjoint.

2.2 The tool-loading oracle

The inner problem—optimal loading for a fixed order—is solved by a greedy rule whose optimality is classical.

Proposition 1 (47)**.** *For a fixed permutation π, an optimal magazine sequence is produced by the* Keep Tool Needed Soonest *(KTNS) policy: whenever a tool must be inserted into a full magazine, evict a tool whose next required use lies furthest in the future (evicting any*

tool never required again). KTNS runs in $O(n|T|)$ time and returns the minimum switch cost over all loading policies for π.

A proof appears in Tang and Denardo [47]; see also Crama et al. [15]. KTNS reduces the SSP to a pure sequencing problem, $Z^* = \min_\pi \mathrm{KTNS}(\pi)$, and is the exact polynomial subproblem exploited by the Benders algorithm of Section 4.2.

Remark 1 (The loading subproblem is offline optimal caching). For a fixed order the loading problem is exactly offline paging: the magazine is a cache of size b, position i requests the set $T_{\pi(i)}$, and an insertion is a cache miss. Proposition 1 is then the tool-switching form of Bélády's optimal replacement rule—evict the item whose next request is furthest in the future—which minimises misses on any fixed request sequence [5]. The contrast with the full problem is instructive: paging fixes the request order, whereas the SSP may *reorder* the jobs, and it is precisely this freedom that turns a polynomially solvable caching problem into an NP-hard one. Every method in this report is, at heart, a way to search over reorderings while delegating the loading to this exact oracle.

KTNS in motion. The rule is easiest to follow on one example, since every algorithm in this report calls it as a black box. Return to the 6-ring processed in the order $1, 2, 3, 4, 5, 6$ (Figure 5). The magazine starts empty and holds $b = 3$ tools. Jobs 1 and 2 need $\{1, 2\}$ and $\{2, 3\}$, so the first magazine is filled with $\{1, 2, 3\}$: this serves both jobs at once and spends the third slot now, on tool 3, to save a switch later. At position 3 job 3 needs $\{3, 4\}$; tool 3 is already loaded, so only tool 4 is inserted—and one resident must leave. Here the rule does its work. Of the residents $\{1, 2, 3\}$, tool 2 is never needed again, whereas tool 1 will be needed by job 6 at the very end; so tool 2 is evicted and tool 1 is kept, sitting idle across four positions. Positions 4 and 5 repeat the pattern—insert the single new tool, evict the one whose next use is furthest away—and position 6 needs $\{6, 1\}$, both already present, so it is free. The tally is three tools for the initial fill and one insertion at each of positions $3, 4, 5$: a switch cost of 6, or 3 once the fill is not charged. That one foresighted choice, holding tool 1 through positions where it does nothing, *is* the entire content of "keep the tool needed soonest"; a myopic policy that evicted tool 1 at position 3 would have to reload it at position 6 and pay more.

2.3 Grouping, sequencing, and the two-phase heuristic

A complementary viewpoint ignores order and asks only which jobs may *share* a magazine.

Definition 2 (Job Grouping Problem)**.** A *group* is a set of jobs $G \subseteq J$ whose combined tool requirement fits in the magazine, $|\bigcup_{j \in G} T_j| \leq b$. The Job Grouping Problem (JGP) partitions J into the minimum number K^* of groups; equivalently, it covers all jobs by the fewest tool configurations of size at most b.

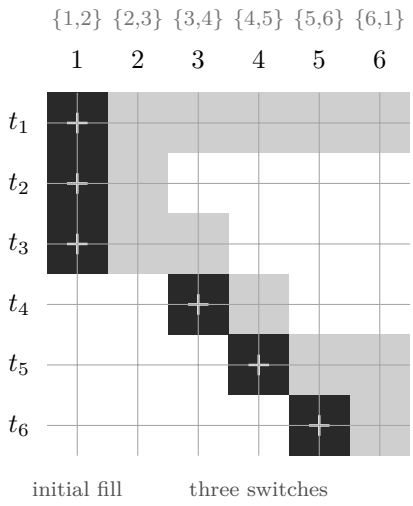

Figure 5: KTNS on the 6-ring, order $1, 2, 3, 4, 5, 6$. Columns are the six processing positions (with the tools each job needs); rows are the six tools. A shaded cell means the tool sits in the magazine at that position; a "+" marks an *insertion*—a switch. Every column holds exactly $b = 3$ tools. Tool 1 (top row) is loaded once and kept across all six positions, idle in the middle, so that it need not be reloaded for job 6: this single act of foresight is what the rule buys. The cost is three insertions to fill the magazine plus three switches thereafter—6 empty-start, 3 free-initial.

The JGP is itself NP-hard and is solved exactly by the asymmetric-representatives formulation of Jans and Desrosiers [27] or by the set-covering column generation of Crama and Oerlemans [14]. It induces the classical two-phase heuristic for the SSP:

1. solve the JGP to obtain K^* groups and a configuration serving each;
2. solve the *Group Sequencing Problem*—order the configurations by a travelling-salesman-type problem whose transition cost is the number of tools that change, a small instance handled by standard solvers [2, 24]—to obtain a sequence of configurations;
3. concatenate the jobs group by group and evaluate the resulting order with KTNS.

We write H for the switch cost this heuristic returns. Since it produces a feasible schedule, $H \geq Z^*$ always; the objects of study are the *gap* $H - Z^*$ and the *ratio* H/Z^* (Section 3).

Example 2 (Grouping the 6-ring). On the running instance the JGP returns $K^* = 3$; one optimal grouping is $\{1, 2\}, \{3, 4\}, \{5, 6\}$, served by the configurations $\{1, 2, 3\}$, $\{3, 4, 5\}$, $\{5, 6, 1\}$. Any two of these three configurations differ in two tools, so every ordering of the groups changes two tools at each of its two transitions, and the best schedule the heuristic produces costs $H = 7$ (free-initial 4). This is one more than the optimum of 6 (free-initial 3) attained in Example 1: a gap of exactly one. The discrepancy is not a failure of the sequencing phase—the groups are ordered optimally—but of the *grouping* phase, which insists on only $K^* = 3$ groups; Section 3 makes this precise.

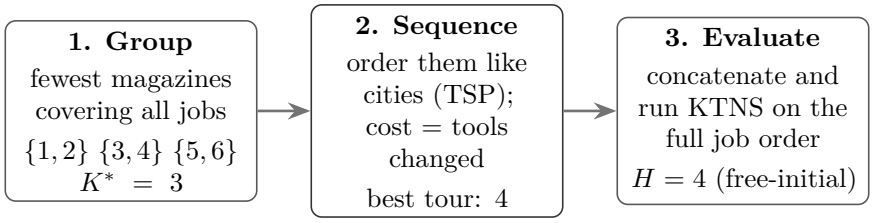

Figure 6: The two-phase heuristic as a pipeline, on the 6-ring. Phase 1 commits to the *fewest* magazines; phase 2 orders them like cities in a travelling-salesman problem; phase 3 flattens the result and scores it with KTNS. Because phase 1 minimises the number of groups before phase 2 can weigh transition costs, the pipeline returns $H = 4$ where the true optimum is $Z^* = 3$—the gap dissected in Section 3.

The heuristic as a pipeline. Figure 6 sets the three phases side by side on the running instance. The first phase discards all ordering information and asks only which jobs can coexist in one magazine, returning the fewest such groups; the second treats each group's configuration as a city and finds the cheapest tour through them, the "distance" between two configurations being the number of tools that differ; the third glues the groups back into a single job order and reads off its true cost with KTNS. The division of labour is deliberate—a hard packing question and a hard ordering question are each handed to a mature, well-understood solver, but it is also the origin of the gap studied in Section 3: the first phase freezes the grouping before the second phase can discover that a slightly larger grouping would have toured more cheaply.

2.4 The configuration–GTSP lens

Both the heuristic and the exact formulations live in the space of configurations.

Definition 3 (Configurations and clusters)**.** A *configuration* is a set $C \subseteq T$ with $|C| = b$. Let $\mathcal{C} = \{C \subseteq T : |C| = b\}$, and for each job j let $H_j = \{C \in \mathcal{C} : T_j \subseteq C\}$ be the *cluster* of configurations able to serve j. For configurations C, C' put $d(C, C') = |C' \setminus C|$, the number of tools inserted in passing from C to C'.

Restricting to full magazines ($|C| = b$) loses no optimality: retaining a tool that may be useful later never increases cost. Under this view a schedule is a walk $C_{(1)}, \ldots, C_{(r)}$ in $\mathcal{C}$ that visits at least one configuration of every cluster H_j, with cost $\sum_l d(C_{(l)}, C_{(l+1)})$; the SSP is therefore a *generalised travelling-salesman problem with overlapping clusters* over $\mathcal{C}$ [12]. The clusters H_j overlap—a rich configuration serves many jobs—which is exactly what distinguishes the SSP from a textbook GTSP with disjoint clusters [20, 38, 42]. That a schedule and such a covering walk have equal cost is immediate in one direction—the distinct consecutive magazines of any schedule form a covering walk of the same cost—and the converse is established as well. It holds in the stronger form ranging over *all* feasible groupings, as Theorem 1 of Section 3 (proof in Appendix A). This lens recurs throughout:

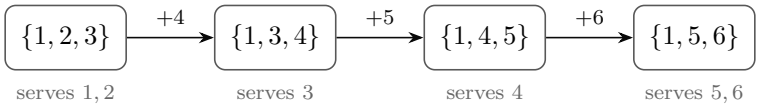

Figure 7: The SSP as a covering walk in configuration space, shown for the optimal schedule of the 6-ring. Each box is a magazine configuration of size $b = 3$; an arrow's label names the single tool swapped in passing to the next, so every transition costs one. The walk must visit a configuration able to serve every job (annotations below). Four configurations, each one tool from the next, realise the optimum $Z^* = 3$. The two-phase heuristic is confined to walks visiting only $K^* = 3$ configurations, which on this instance cannot hold every transition down to a single tool—the source of the gap analysed in Section 3.

it underlies the gap analysis of Section 3, the position-indexed formulations of Section 4.4, and the reading of the Benders master as a travelling-salesman skeleton (Section 4.2).

2.5 Two structural lower bounds

The following bounds are elementary but pervasive: they drive the gap theory of Section 3, calibrate the linear relaxations of Section 4, and serve as correctness checks for the computational study. Both are stated in the *free-initial* cost (Section 2.6), which does not charge the first magazine fill; the conversion to Definition 1 is the additive constant of Proposition 4.

Proposition 2 (Grouping bound)**.** $Z^* \geq K^* - 1$.

Proof. Fix any schedule and list its magazine states $M_1, \ldots, M_n$; collapse each maximal run of equal states to a single representative, obtaining a sequence $C_{(1)}, \ldots, C_{(r)}$ in which consecutive entries differ. Every job is served by some state, hence by some $C_{(l)}$, so $\{C_{(1)}, \ldots, C_{(r)}\}$ is a family of sets of size at most b covering all jobs. Assigning each job to one configuration that contains its requirement partitions J into groups, one per *distinct* configuration, each group having combined requirement contained in that configuration and hence of size at most b; this is a feasible grouping, so the number of distinct configurations is at least K^*. Therefore $r \geq K^*$. Finally, each of the $r - 1$ consecutive transitions changes at least one tool, so the free-initial switch cost is at least $r - 1 \geq K^* - 1$. □

Proposition 3 (Coverage bound)**.** $Z^* \geq |U| - b$.

Proof. Each tool of U is required by some job and therefore occupies the magazine at some position, so it is inserted at least once unless it belongs to the initial fill, which contains at most b tools. Hence at least $|U| - b$ insertions occur in the free-initial count. □

Corollary 1. $Z^* \geq \max\{K^* - 1,\ |U| - b\}$.

The two bounds are individually tight on some families and simultaneously slack on others (Section 3). The coverage bound $|U| - b$ is exactly the value to which the multicommodity-flow relaxation of da Silva and Yanasse [17] collapses—a fact central to the assessment of Section 5.3. The two measure different things: on the 6-ring, $K^* - 1 = 2$ while $|U| - b = 6 - 3 = 3$, so the coverage bound is tight ($Z^* = 3$ free-initial, Example 1) and the grouping bound is slack, whereas on instances built from many nearly tool-disjoint groups the reverse holds. Neither dominates in general, and their maximum, though always valid, is not always attained: over 21,569 enumerated instances with jobs of sizes $\{2, 3\}$ at $b = 3$, 2,410 have Z^* strictly above both bounds, and the 8-job sliding-window ring at $b = 4$ ($T_j = \{j, j + 1, j + 2\}$ modulo 8, $|U| = 8$) has $Z^* = 5 > \max(3, 4)$.

2.6 Switch-cost convention

Formulations in the literature charge the initial magazine fill differently, and comparing their objective values naively produces spurious discrepancies; we therefore fix the convention.

Definition 4 (Empty-start and free-initial costs)**.** The *empty-start* cost charges every insertion from an initially empty magazine ($M_0 = \varnothing$, as in Definition 1). The *free-initial* cost does not charge the first fill; equivalently, it subtracts the $\min(b, |U|)$ tools loaded at the first position.

Proposition 4 (Convention identity)**.** *For every schedule,* $cost_{\text{empty}} = cost_{\text{free}} + \min(b, |U|)$.

Proof. An optimal loading (Proposition 1) keeps the magazine full, so the first position loads $\min(b, |U|)$ tools; the empty-start count charges these initial insertions while the free-initial count, by definition, omits them, and every later insertion is counted identically by both. □

The shift is a per-instance constant, so the gap $H - Z^*$ is convention-invariant whereas the ratio H/Z^* is not; ratio statements below are always taken in a single fixed convention. When objective values from different formulations are compared, they are first reduced to the empty-start convention via Proposition 4.

2.7 Related work

Origins: the loading rule, and the hardness of ordering. The problem was posed by Tang and Denardo [47], who isolated its two-level structure and settled the inner level once and for all: for a fixed job order the optimal loading is the Keep-Tool-Needed-Soonest rule, proved optimal in $O(n|T|)$ time. Their paper is also the source of two facts this report leans on—the pairwise bound $\max(0, |T_i \cup T_j| - b)$ on the switches forced between two adjacent jobs, which seeds our Benders master, and the observation that their own

integer formulation has a linear relaxation *identically zero*, so that ordering, not loading, carries all the difficulty and resists the linear-programming machinery from the very start. That difficulty was made precise by Crama et al. [15], who proved the SSP $\mathcal{NP}$-hard for every fixed capacity $b \geq 2$ by a reduction to a Hamiltonian-path problem on an auxiliary edge graph, fixed the seven modelling assumptions now standard in the field, and gave an interval-matrix proof of KTNS's optimality that extends to arbitrary per-tool switching costs. A later study [16] draws the boundary of tractability sharply: once tools occupy several magazine slots the loading problem itself becomes strongly $\mathcal{NP}$-hard (by a 3-partition reduction), yet for a fixed capacity it stays polynomial through a shortest-path model—so the uniform SSP, whose loading is the paging problem of Remark 1 [5], sits at the tractable edge of a family that turns intractable the instant the tools stop being uniform.

The exact-formulation lineage, and where it plateaus. The exact side of the field is a two-decade effort to give the ordering problem a linear relaxation strong enough to drive branch-and-bound, and its trajectory is the single most important piece of context for this report: every rung of the ladder ends at the same quantity the structural theory singles out. Laporte et al. [29] replaced the Tang–Denardo model—whose relaxation is useless—with a tool-state formulation on a travelling-salesman skeleton (a start/end depot, arc variables, magazine-state and switch variables), strengthened it with three families of valid inequalities, and solved it two ways: a linear-programming branch-and-cut seeded by the GENIUS heuristic [25], and a branch-and-*bound* carrying no linear relaxation at all but two combinatorial bounds on partial sequences, in the enumerative spirit later formalised by Yanasse et al. [50]. The branch-and-bound reaches 25-job instances while the branch-and-cut stalls near eight—an early, concrete warning that on this problem the relaxation, not the search, is the weak link. Ghiani et al. [22] recast the SSP as a least-cost Hamiltonian cycle with a nonlinear arc cost, exploited the symmetry that a sequence and its reversal cost the same to halve the search, and separated comb inequalities with dynamically updated KTNS bounds, again pushing the reachable size upward. Catanzaro et al. [9] then produced the tightest *compact* models to date—arc-based formulations whose relaxation provably dominates Laporte's, a total-unimodularity argument recovering KTNS-optimality from linear-programming duality, and a reading of an optimal sequence as one minimising the "zero-blocks" of the tool-incidence matrix—and, in the same paper, the asymmetric-representatives MILP for the Job Grouping Problem that this codebase uses. Most recently da Silva and Yanasse [17] gave a multicommodity-flow model that needs no lazy constraints and solves some 17% more of the standard instances than any predecessor, and whose relaxation they prove equals exactly $|U| - b$. That last fact is the ceiling. The strongest compact relaxation produced in twenty years coincides with the elementary coverage bound of Proposition 3—the very quantity Section 3 shows to be

the optimum on a large part of the instance space and a strict underestimate on the rest. The compact literature has, in effect, climbed to the coverage bound and stopped; the two methods of Section 4 are chosen precisely to get past it.

The configuration view: grouping, GTSP and column generation. A parallel line abandons job positions for tool *configurations*. Crama and Oerlemans [14] formulate the Job Grouping Problem as a set-covering model solved by column generation, whose relaxation is strong and whose pricing subproblem is a Set-Union Knapsack Problem [23]—$\mathcal{NP}$-hard even under restrictive conditions, a difficulty that returns as the bottleneck of the position-formulation pricers of Section 4.4; Jans and Desrosiers [27] give the compact asymmetric-representatives alternative. The configuration view that unifies grouping and sequencing is developed by Colares and Wagler [12], Colares et al. [13], who cast the SSP as a generalised travelling-salesman problem whose clusters—the configurations able to serve each job—*overlap*, solved by a non-compact integer program with exponentially many transition variables and, in progress, by column generation. The overlap is exactly what separates it from the textbook GTSP with disjoint clusters surveyed by Pop et al. [42], whose Noon–Bean reductions and branch-and-cut do not transfer directly; and where such pricing spawns constraints that depend on the generated columns, the simultaneous column-and-row framework of Muter et al. [37] is the relevant machinery. Two recent theses sit closest to this work: Otiai [39] builds an exact branch-and-price for the Job Grouping Problem with Ryan–Foster branching and finds its set-covering relaxation strong enough to close in seconds instances the compact model cannot, while Moreira [36] studies the SSP through the magazine-configuration formulation and measures its relaxation against the compact models—measurements we revisit in Section 5.3. Our branch-and-price follows the standard column-generation apparatus [4, 31, 49], branching by the same/different rule of Ryan and Foster [46] and eliminating subtours with the constraints of Miller et al. [34] lifted as in Desrochers and Laporte [18].

Heuristics and metaheuristics. Because the exact methods scale poorly, most of the SSP literature is heuristic, and its trajectory is itself evidence for the thesis of this report. Early work pairs a constructive, travelling-salesman-style rule with local search over the job order [15]; the GENI and GENIUS heuristics of Hertz et al. [25] sharpen those constructions and, as a byproduct, furnish the upper bound inside Laporte's branch-and-cut, while enumerative schemes with dominance pruning [50] and the bounding arguments of Ghiani et al. [22] complete the pre-metaheuristic era. The current state of the art is the hybrid genetic search of Mecler et al. [33], which couples an order-based genetic algorithm with local search and seeds its population from Job Grouping solutions, so that the grouping view is built into even the best heuristic. The recurring empirical finding across two decades of this work is that grouping-based starting points are hard

to beat; that is precisely the phenomenon Section 3 explains and, for the first time, bounds. Read together with the exact literature, the picture is a field that has converged on the group-then-sequence idea from both directions—as a heuristic and as an exact skeleton—without ever quantifying what it costs, which is the gap this report sets out to close.

The decomposition in practice. The group-then-sequence decomposition is not merely a theoretical device. In colour printing the SSP appears verbatim—ink cartridges are the tools, the cartridge rack the magazine—and Burger et al. [7] solve the industrial problem by exactly this decomposition, modelling the grouping as unicost set covering and the sequencing as a travelling-salesman problem, enumerating optimal groupings by Stirling-number arguments, and measuring the decomposition's suboptimality empirically: it is small on their instances, consistent with earlier reports they survey. What the literature does not contain is a *worst-case* analysis of the decomposition—bounds on its gap or ratio, extremal instances, or conditions for exactness. To our knowledge no worst-case guarantee has been published for *any* SSP heuristic; the decomposition in particular had only ever been assessed empirically. Section 3 supplies that analysis.

Recent exact developments. The exact frontier has moved again in the last two years, by routes that reinforce rather than contradict the reading above. Akhundov and Ostrowski [1] exploit the reversal symmetry of Ghiani et al. [22] systematically, deriving symmetry-broken position-based and arc-flow formulations from a job-group representation and solving larger instances than the earlier compact models. Legrand et al. [30]—with Catanzaro among the authors—abandon integer programming altogether for a dynamic-programming and A^* search equipped with a new family of lower bounds *provably tighter* than any in the prior literature, reporting the strongest exact results to date and outperforming both branch-and-bound and the ILP formulations. Their bounds are combinatorial, computed inside the search rather than as a linear relaxation, and that is exactly the moral: the way past the coverage ceiling has been to leave the compact-LP paradigm—whether for a search with sharper combinatorial bounds (Legrand et al.), for a decomposition whose subproblem is solved exactly (the Benders route of Section 4.2), or for a configuration relaxation engineered to exceed $|U| - b$ (the position–transition route of Section 4.4). The position formulations here are, to our knowledge, the first *linear relaxation over configurations* to break the bound; they are complementary to, not in competition with, the search bounds of Legrand et al. [30], and a head-to-head comparison of the two on shared instances is a natural next step. Activity has also broadened to sequence-dependent and non-identical parallel-machine variants with new mixed-integer models, but the single-machine switch-minimisation core studied here remains the object of the sharpest exact competition.

Year	Work	Approach	Key advance
1988	Tang and Denardo [47]	ILP; KTNS rule	KTNS optimal for a fixed order; LP relaxation = 0
1994	Crama et al. [15]	complexity; heuristics	SSP $\mathcal{NP}$-hard; standard model fixed
2004	Laporte et al. [29]	LSS ILP; B&C and B&B	exact to 25 jobs (B&B beats B&C)
2010	Ghiani et al. [22]	Hamiltonian cycle	reversal symmetry halves the search
2015	Catanzaro et al. [9]	tighter compact F3/F4	LP relaxation dominates LSS
2024	da Silva and Yanasse [17]	multicommodity flow	LP = $\lvert U\rvert - b$; +17% solved
2024	Akhundov and Ostrowski [1]	symmetry; arc-flow	symmetry-broken position/flow models
2025	Legrand et al. [30]	dynamic programming, A^*	combinatorial bounds tighter than any LP
–	this work	Benders; position B&P	past the $\lvert U\rvert - b$ ceiling by decomposition

Table 1: The exact-method lineage for the SSP. Two decades of compact formulations tightened the linear relaxation only up to the coverage bound $|U| - b$ [17]; moving past it has required leaving the compact-LP paradigm—by search with combinatorial bounds [30], or by the decomposition and configuration-relaxation routes of this report.

Benders decomposition and its branch-and-cut integration. The solver of Section 4.2 inherits a well-developed methodology. Rahmaniani et al. [45] survey the acceleration of Benders decomposition; the particular setting in which the subproblem is settled not by linear-programming duality but by a combinatorial oracle is the logic-based Benders of Hooker and Ottosson [26] and the combinatorial Benders cuts of Codato and Fischetti [11]. The immediate template, however, is the recent study of Kashou et al. [28], who apply branch-and-Benders-cut to a robust production-line balancing problem: they integrate Benders into the branch-and-bound tree through lazy-constraint callbacks, attack the dual-subproblem bottleneck with four tailored algorithms replacing a general solver inside the Benders loop, and further strengthen the method with a heuristic that pre-injects optimality cuts and a reformulated master—and they report that even so the decomposition does not systematically beat the non-decomposed formulation. That honest negative is precisely what recommends the approach *here*, for the SSP occupies the regime their assembly-line problem lacked: its dual subproblem is not a general program needing tailored algorithms but the *polynomially* solvable KTNS oracle, so the loading cut is exact and essentially free at every callback. The present solver adopts the same architecture—lazy-callback branch-and-Benders-cut, a heuristic warm start with seeded cuts, and a strengthened master—in the one setting where a Benders decomposition is structurally expected to pay off rather than merely break even.

What remains open, and the route taken here. Three observations from the preceding paragraphs set the agenda. First, the decomposition heuristic has a strong empirical record but no worst-case theory; Section 3 builds one. Second, on the exact side

the strongest compact relaxation equals the elementary coverage bound $|U| - b$ [17], and the configuration relaxation collapses to the same value on most instances [36]; progress therefore requires either a method that does not lean on the linear relaxation—the Benders route of Section 4.2, whose strength is an exact polynomial subproblem—or a formulation whose relaxation can exceed the bound, the position–transition route of Section 4.4. Third, the configuration view that is so effective for the JGP [39] lacks a tractable exact SSP counterpart, the natural model carrying exponentially many corrected subtour constraints (Section 4.4); that is the opening the position-indexed formulations address.

3 Structural analysis of the heuristic gap

The group-then-sequence heuristic is the standard practical method for the SSP, and where it has been tested against exact optima its measured suboptimality is small [7]; a worst-case analysis, however, does not exist. The source of the gap is that the heuristic commits to a *minimum-cardinality* grouping before it sequences, whereas the true optimum may be served by a larger grouping with cheaper transitions; the gap is precisely the difference between the two. This is what the running instance shows (Example 2), and the coverage bound $|U| - b$ that will limit the exact methods of Section 4 reappears here as the value that makes the gap vanish. The section proceeds in six steps. The gap is first identified exactly: it is the price of minimality (Section 3.1). It is then shown to be real and unbounded (Section 3.2). Worst-case guarantees are established for every instance (Section 3.3), and the extremal cases are worked out at $K^* = 3$, including the walk-model certificate against the candidate condition $K^* - 2$ and its reopening for the KTNS-evaluated heuristic (Section 3.4). The combinatorial structure that can and cannot decide the gap is characterised (Section 3.5). Finally, a setup-cost spectrum shows when the whole question collapses (Section 3.6), and the algorithmic question the analysis leaves open is recorded (Section 3.7).

Before the formal development it helps to see the mechanism concretely. The optimum of the 6-ring is a walk through four configurations, each one tool from the next (Figure 7); the heuristic, restricted to the three minimum-cardinality groups, is stranded among configurations that pairwise differ in *two* tools (Figure 8). No ordering of three such configurations can match the four-configuration walk, and the resulting unit of lost efficiency is the gap. The theory below shows this picture is general: the gap is always the difference between the best walk over *all* groupings and the best walk over minimum-cardinality ones, and it vanishes exactly when some smallest grouping already admits a cheapest walk.

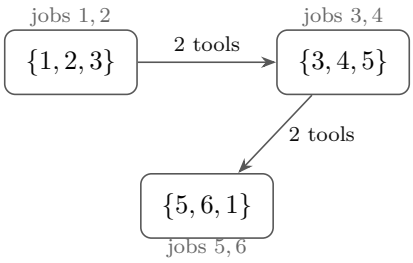

Figure 8: Why the ring loses. The heuristic may use only the $K^* = 3$ minimum-cardinality groups $\{1,2\}\{3,4\}\{5,6\}$, forcing the three magazines $\{1,2,3\}, \{3,4,5\}, \{5,6,1\}$; *every* pair of them differs in two tools. Sequencing three such configurations pays two switches on each of its two transitions, so the best the heuristic achieves is $H = 4$ (free-initial), against the optimum $Z^* = 3$ of Figure 7. The loss is not a sequencing error—the groups are ordered perfectly—but the price of using only three groups.

3.1 The gap is the price of minimum-cardinality grouping

We attach a cost to a grouping by ordering its configurations optimally.

Definition 5 (Feasible grouping and its cost)**.** A *feasible grouping* is a partition $\mathcal{P} = \{G_1, \ldots, G_p\}$ of J together with, for each group, a configuration $C_l \supseteq \bigcup_{j \in G_l} T_j$ (which exists iff $|\bigcup_{j \in G_l} T_j| \leq b$). Its *cost* is the cheapest free-initial walk through its configurations,

$$\gamma(\mathcal{P}) = \min_{\sigma \in S_p} \sum_{l=1}^{p-1} d\Big(C_{\sigma(l)}, C_{\sigma(l+1)}\Big).$$

The Job Grouping Problem seeks a feasible grouping of minimum cardinality K^*, and the two-phase heuristic evaluates γ on such a grouping. Removing the cardinality restriction makes the framework exact.

Theorem 1 (Grouping exactness)**.** $Z^* = \min_{\mathcal{P}} \gamma(\mathcal{P})$*, the minimum taken over* all *feasible groupings, of every cardinality.*

The proof (Appendix A) is a pair of constructions: any feasible grouping, processed group by group, is a schedule of cost $\gamma(\mathcal{P})$, giving "$\leq$"; and any optimal schedule, its jobs grouped by the magazine under which they run, is a feasible grouping of no greater cost, giving "$\geq$".

Corollary 2 (The heuristic gap)**.** *Writing* $H = \min\{\gamma(\mathcal{P}) : |\mathcal{P}| = K^*\}$ *for the best two-phase cost,*

$$H - Z^* = \min_{|\mathcal{P}| = K^*} \gamma(\mathcal{P}) \; - \; \min_{\mathcal{P}} \gamma(\mathcal{P}) \; \geq \; 0,$$

and the gap is strictly positive exactly when no minimum-cardinality grouping attains the overall minimum of γ*.*

Remark 2 (A zero-gap certificate). Since $H \geq Z^* \geq \max(K^* - 1, |U| - b)$ (Corollary 1), any instance on which the heuristic attains a lower bound, $H = \max(K^* - 1, |U| - b)$, has zero gap. This already disposes of the k-rings with $k \in \{3, 4, 5\}$ (Table 2), where $H = |U| - b$.

Example 3 (Where the ring loses). The optimum of the 6-ring is realised by the *four*-configuration grouping $\{1, 2, 3\} \to \{1, 3, 4\} \to \{1, 4, 5\} \to \{1, 5, 6\}$, consecutive configurations differing in a single tool, for $\gamma = 3$ (cf. Example 1). Every minimum-cardinality grouping uses only $K^* = 3$ configurations and costs 4 (Example 2). The gap of one is precisely the difference in Corollary 2: the cheaper grouping is not of minimum cardinality, so the grouping phase never offers it to the sequencer.

3.2 The gap is real and unbounded

Corollary 2 locates the gap; the next question is whether it actually occurs, and how large it can grow. Table 2 collects the k-ring ($T_j = \{j, j + 1\}$ modulo k, $b = 3$) for $k = 3, \ldots, 8$: the gap first appears at $k = 6$, where the ratio is largest. Replicating the worst ring makes

k	K^*	Z^*	H	gap	H/Z^*
3	1	0	0	0	—
4	2	1	1	0	1
5	3	2	2	0	1
6	3	3	4	1	4/3
7	4	4	5	1	5/4
8	4	5	6	1	6/5

Table 2: The k-ring ($b = 3$, free-initial costs).

the gap arbitrarily large.

Proposition 5 (Unbounded gap)**.** *Let R_g be g tool-disjoint copies of the 6-ring ($b = 3$, $|U| = 6g$). Then $Z^* = 6g - 3$ and $H = 7g - 3$, so the gap $H - Z^* = g$ is unbounded while the ratio $(7g - 3)/(6g - 3)$ decreases from $4/3$ at $g = 1$ towards $7/6$.*

The proof (Appendix A) pairs the coverage bound $Z^* \geq |U| - b = 6g - 3$ with a sequential schedule attaining it, and counts the heuristic's cost as g within-copy contributions of 4 and $g - 1$ inter-copy transitions of 3.

Remark 3 (Perfectness does not predict the gap). One might hope that polyhedral regularity of an instance governs its gap; it does not. Form the *conflict graph* on the jobs, joining i and j when they cannot share a configuration ($|T_i \cup T_j| > b$). For the 6-ring this graph is the triangular prism, the complement of C_6, which is perfect, yet the gap is one; for the 5-ring it is the odd cycle C_5, which is imperfect [10], yet the gap is zero. Perfectness of the conflict graph and the size of the gap are therefore independent.

3.3 Worst-case guarantees

The previous subsection showed the gap can grow without bound in absolute terms; this one shows it is nonetheless tightly controlled, by bounds that hold for *every* instance and every capacity—not only for the ring families that witness the extremes. Two quantities recur and are worth naming now: $q = |U| - b$, the excess of used tools over the capacity (the coverage bound), and later R_{opt}, the number of *re-insertions* an optimal schedule makes (tools that leave the magazine and come back). Throughout, $K^* \geq 2$ (at $K^* = 1$ the heuristic is trivially exact) and costs are free-initial.

Lemma 1 (Cost identity)**.** *Restrict every magazine state to a subset of U (always possible when $K^* \geq 2$, and never worse). The free-initial cost of any full-magazine schedule then equals $q + R$, where $R \geq 0$ counts* re-insertions*: insertions of tools that were present earlier and evicted.*

Proof. Each of the q tools of U outside the initial load is required by some job, so it is inserted at least once; splitting all insertions into first-time ones (exactly q) and the rest (by definition R) gives the identity. □

Hence $Z^* = q + R_{\text{opt}}$ and $H = q + R_H$, so the gap $H - Z^* = R_H - R_{\text{opt}}$ is *exactly the excess re-insertions* that the minimum-cardinality grouping forces. The 6-ring's single extra switch is one such re-insertion: tool t_1, needed in the first and the last group but not in the middle one, is evicted at a group boundary and reloaded, where the optimal schedule keeps it resident throughout (Example 3).

Lemma 2 (Transition cap)**.** *In any ordering of configurations contained in U, every transition inserts at most* $\min(b,\, q)$ *tools; hence $H \leq (K^* - 1)\min(b,\, q)$.*

Proof. A transition into C_{k+1} inserts $|C_{k+1} \setminus C_k| \leq |C_{k+1}| = b$ tools, and also at most $|U \setminus C_k| = q$ tools, since $C_{k+1} \subseteq U$ and $|C_k| = b$. A covering configuration inside U exists for every group, so some minimum-cardinality grouping uses only such configurations, and H is at most its cost. □

These two lemmas already yield the report's central guarantee. Lemma 2 caps the heuristic's absolute cost and Corollary 1 floors the optimum; dividing the one by the other bounds how far from optimal the heuristic can stray—by a factor that, and this is the useful part, shrinks on its own as the instance grows simpler, whether "simpler" means few groups, a small tool excess q, or a small magazine.

Theorem 2 (Worst-case guarantee)**.** *For every instance with $K^* \geq 2$,*

$$\frac{H}{Z^*} \;\leq\; \frac{(K^* - 1)\min(b,\, q)}{\max(K^* - 1,\; q)} \;\leq\; \min\Big(b,\; K^* - 1,\; q\Big).$$

Proof. Divide Lemma 2 by $Z^* \geq \max(K^* - 1, q) \geq 1$ (Corollary 1). For the second inequality, $(K^* - 1)\, q / \max(K^* - 1, q) = \min(K^* - 1, q)$ and $\min(b, q) \leq q$ give the $\min(K^* - 1, q)$ part, while $(K^* - 1)/\max(K^* - 1, q) \leq 1$ and $\min(b, q) \leq b$ give the b part. □

This subsumes the immediate b-approximation ($H \leq b(K^* - 1)$ divided by $Z^* \geq K^* - 1$): the guarantee tightens automatically on instances with few groups or with few tools in excess of the capacity.

Corollary 3 (Zero-gap classes, every b)**.** *(i) If $K^* = 2$ the gap is zero; indeed $H = Z^* = q$. (ii) If $|U| \leq b + 1$ the gap is zero. (iii) If $K^* = 3$ then $H/Z^* \leq 2$.*

Proof. All three are instances of Theorem 2, whose ratio bound is $K^* - 1 = 1$, $q \leq 1$, and $K^* - 1 = 2$ respectively. For the equality in (i), pad the first group's configuration to size b with tools of the second group's requirement and pad the second configuration from the first; the single transition then inserts exactly the q tools of U missing from the first configuration, so $H \leq q \leq Z^* \leq H$ by the coverage bound. □

Worst cases therefore require $K^* \geq 3$—consistent with every positive-gap instance found in enumeration having $K^* \in \{3, 4\}$.

The theorem's cap is far above the worst case actually observed. Exhaustive enumeration supports two sharper statements.

Conjecture 1 ($b = 3$ ratio)**.** For every instance with $b = 3$, $H/Z^* \leq 4/3$.

Checked by exhaustive enumeration of the $b = 3$ two-tools-per-job family with $m \leq 6$ (10,691 instances) and of mixed size-$\{2, 3\}$ jobs with $m \leq 5$ (a further 6,065): the maximum ratio is exactly $4/3$, attained at the 6-ring, with no counterexample.

3.4 The exact worst case at $K^* = 3$

The guarantees above say how large the gap *cannot* be; this subsection asks what it *is* in the worst case, starting where every known positive-gap instance lives: $K^* = 3$. The answer has three parts. First, an exact formula for the heuristic cost, which reduces the whole question to the overlaps of three configurations. Second, quantitative bounds that every known extremal instance attains with equality. Third, a certificate and a separation: the natural candidate condition gap $\leq K^* - 2$—consistent with all evidence ever collected, which happened to lie entirely at $b \leq 4$—is proved on wide regimes and then shown *false* exactly at the first parameters our theorems leave open. What remains afterwards is not a broken conjecture but a sharper question: whether the bound below is the exact worst case.

Proposition 6 (Exact heuristic cost at $K^* = 3$)**.** *If $K^* = 3$ then, with configurations restricted to subsets of U,*

$$H \;=\; q \;+\; \min\;\min\Big(x_{ab},\, x_{bc},\, x_{ca}\Big), \qquad x_{uv} = |(C_u \cap C_v) \setminus C_w|,$$

the outer minimum over feasible 3-groupings and configuration choices. Hence the gap equals $\min\{\cdot\} - R_{\mathrm{opt}}$, and it vanishes exactly when some minimum-cardinality grouping admits configurations whose smallest pairwise overlap outside the third is at most R_{opt}.

Proof. The three configurations of a feasible 3-grouping are distinct (equal ones would merge two groups, contradicting $K^* = 3$) and cover U. For a path (C_a, C_b, C_c), splitting the cost into first-time insertions and re-insertions (Lemma 1) gives cost $q + |(C_a \cap C_c) \setminus C_b|$: the re-inserted tools are exactly those of C_c present in C_a but absent from C_b. Minimising over the three orderings selects the cheapest middle configuration, and H minimises over groupings and configuration choices. □

This turns grouping selection at $K^* = 3$ (Section 3.7) into an explicit criterion: seek the grouping minimising the smallest pairwise configuration overlap outside the third.

Lemma 3 (Overlap counting)**.** *Any three configurations $C_a, C_b, C_c \subseteq U$ of size b with $C_a \cup C_b \cup C_c = U$ satisfy $\min(x_{ab}, x_{bc}, x_{ca}) \le \lfloor (2b - q)/3 \rfloor$.*

Proof. Count tools by the number of configurations containing them: with a tools in exactly one configuration, $\sum x := x_{ab} + x_{bc} + x_{ca}$ in exactly two, and c_3 in all three, the tool count gives $a + \sum x + c_3 = b + q$ and the degree sum gives $a + 2\sum x + 3c_3 = 3b$; subtracting, $\sum x = 2b - q - 2c_3 \le 2b - q$. The minimum is at most the mean $\sum x/3$, and it is an integer. □

Proposition 7 (The gap at $K^* = 3$: quantitative bound and gap ≤ 1 regimes)**.** *Let $K^* = 3$ and $R_{\mathrm{opt}} = Z^* - q$. Then*

$$\mathrm{gap} \;\le\; \max\Big(0,\; \min\Big(q,\; \lfloor (2b - q)/3 \rfloor\Big) - R_{\mathrm{opt}}\Big),$$

and the bound is attained by the 6-ring and by the $b = 4$ extremal witness. Consequently gap ≤ 1 *holds (i) for* every *instance with $b \le 4$; (ii) whenever $2b \le q + 3R_{\mathrm{opt}} + 5$; (iii) whenever $R_{\mathrm{opt}} \ge q - 1$; and (iv) at $Z^* = q = 3$ also when two of the four walk groups have combined requirement fitting one magazine. At $K^* = 3$ the bound* gap ≤ 1 *can therefore fail only in the corner $2b \ge q + 3R_{\mathrm{opt}} + 6$ with $R_{\mathrm{opt}} \le q - 2$—necessarily $b \ge 5$—and Proposition 9 shows that there it does fail.*

Proof. The configurations of any feasible 3-grouping (one exists) cover U, so Proposition 6 with Lemma 3 gives $H \le q + \lfloor (2b - q)/3 \rfloor$, while Lemma 2 gives $H \le 2\min(b, q) \le q + q$; subtracting $Z^* = q + R_{\mathrm{opt}}$ yields the display. For (i) at $b = 3$: $\lfloor (6 - q)/3 \rfloor \le 1$ for all

$q \geq 1$. At $b = 4$: the display is ≤ 1 for $q \geq 3$; for $q = 2$, either $R_{\text{opt}} \geq 1$ (display ≤ 1) or $Z^* = q \leq 2$, in which case the gap vanishes (Appendix A); $q \leq 1$ is Corollary 3(ii). Parts (ii) and (iii) read off the display; part (iv) is proved in Appendix A. □

Proposition 8 (A quantitative bound for every K^*)**.** *For every instance with $K^* \geq 2$,*

$$\text{gap} \;\leq\; \max\Bigl(0,\; \Bigl\lfloor \frac{q\,\bigl(b(K^*-1)-q\bigr)}{b+q} \Bigr\rfloor - R_{\text{opt}}\Bigr).$$

Proof. Fix a minimum-cardinality grouping; its configurations $C_1, \ldots, C_{K^*} \subseteq U$ are pairwise distinct (else two groups would merge) and cover U. In an ordering σ the path cost is $q + \sum_t(\text{runs}_\sigma(t) - 1)$, where $\text{runs}_\sigma(t)$ counts the maximal blocks of consecutive configurations containing t: every block start after the first is a re-insertion (Lemma 1). Let d_t be the number of configurations containing t, so $\sum_t d_t = K^* b$ over $b + q$ tools. Under a uniformly random ordering $\mathbb{E}[\text{runs}(t) - 1] = (d_t - 1)(K^* - d_t)/K^*$; this is concave in d_t, so by Jensen the expected total excess is at most $q\,(b(K^* - 1) - q)/(b + q)$. Some ordering achieves at most the mean, the total is an integer, and subtracting $Z^* = q + R_{\text{opt}}$ gives the claim. □

At $K^* = 3$ this and Proposition 7 are incomparable, and both are tight at the 6-ring. The bound grows linearly in K^*; it is unconditional and, to our knowledge, the first quantitative worst-case bound on the decomposition valid for every K^*—on the $b = 5$, $K^* = 4$ walk-model witness it gives 4 where the transition cap gives 8.

Corollary 4 (Small optima are gap-free)**.** *If $Z^* \leq 2$ then the gap is zero, for every b and every K^*.*

Proof. $Z^* \geq q$, so $q \leq 2$. If $q \leq 1$ the gap vanishes by Corollary 3(ii). If $q = 2$ then $Z^* = q$, and the walk argument of Appendix A yields at most $q + 1 = 3$ configurations, each serving a job, with $K^* \leq p$; for $K^* \leq 2$ the gap vanishes by Corollary 3(i), and for $K^* = 3$ necessarily $p = K^*$, which forces $H = Z^*$. □

Corollary 5 ($Z^* \leq 3$ forces gap at most one)**.** *If $Z^* \leq 3$ then $H - Z^* \leq 1$, for every b and every K^*.*

Proof. Write $Z^* = q + R_{\text{opt}}$, so $q \leq 3$. If $q \leq 1$ the gap is zero (Corollary 3(ii)), and if $K^* \leq 2$ likewise (Corollary 3(i)); assume $q \in \{2, 3\}$ and $K^* \geq 3$. The excised optimal walk has $p \leq Z^* + 1 \leq 4$ configurations, each serving a job, and exhibits a feasible p-grouping, so $K^* \leq p \leq 4$. If $K^* = 4$ then $p = 4 = K^*$: the walk's grouping is of minimum cardinality and its cost is Z^*, forcing $H = Z^*$. There remains $K^* = 3$. For $(q, R_{\text{opt}}) = (3, 0)$ the merge argument in the proof of Proposition 7 (Appendix A) gives gap ≤ 1. For $(q, R_{\text{opt}}) = (2, 1)$ the bound of Proposition 7 reads $\max\bigl(0, \min(2, \lfloor(2b - 2)/3\rfloor) - 1\bigr) \leq 1$. □

In exhaustive sampling the gap is in fact 0 whenever $Z^* \leq 3$; whether the value 1 is attainable (necessarily at $K^* = 3$) is open.

The heuristic admits two readings, and separating them is essential here. Write H_{walk} for its cost when each group's configuration is held unchanged for the whole group—the walk value γ minimised over minimum-cardinality groupings—and H for the value of the method as actually defined, whose final step re-evaluates the concatenated job order with KTNS. Always $H \leq H_{\text{walk}}$, so every upper bound proved above for H_{walk} is valid for H; the two can differ, and they differ precisely on the extremal instances below.

Proposition 9 (The $K^* - 2$ bound fails for the walk value)**.** *There are instances with $K^* = 3$ and $H_{\text{walk}} - Z^* = 2$. One, with $b = 5$ and $U = \{0, \ldots, 8\}$:*

$$T = \Big(\{0,1,2,4,7\},\ \{0,4,5\},\ \{1,5\},\ \{2,6,8\},\ \{3,4,5,8\}\Big),$$

has $q = 4$, $Z^* = 4$ and $H_{\text{walk}} = 6$ (both values exhaustive), hence $H_{\text{walk}} - Z^* = 2 > K^* - 2$. The instance sits exactly on the boundary of the regimes of Proposition 7, $2b = q + 3R_{\text{opt}} + 6$, and attains that proposition's bound with equality: $\min(q, \lfloor (2b - q)/3 \rfloor) - R_{\text{opt}} = 2$. For the heuristic itself, however, the KTNS step beats the walk on this very witness: exhaustive evaluation gives $H = 5$, a gap of 1—*and the same collapse to gap* 1 *occurs on every walk-extremal instance known.*

Two consequences. First, the quantitative bounds of Propositions 7 and 8 are exact for the walk value at every known walk-extremal instance—the 6-ring at $b = 3$, the $b = 4$ witness, the $b = 5$ witnesses, and further attainment points at $(b, q, R_{\text{opt}}) = (5, 2, 1)$ and $(5, 3, 1)$—while at the corners $(5, 3, 0)$ and $(6, 3, 0)$ directed search finds only H_{walk}-gap 1 against predicted caps of 2 and 3. Second, and more importantly, the candidate condition gap $\leq K^* - 2$ is *reopened* for the heuristic itself: no instance with $H - Z^* > K^* - 2$ is known—directed hunts across $b \in \{3, 4, 5\}$ and $K^* \in \{3, 4, 5\}$, including perturbation searches around the walk-extremal witnesses, find H-gap at most 1 in every sampled (b, K^*, q, R) cell—and the question of the exact worst case must now be asked of H, where the KTNS rule's advantage over the walk is itself a structural phenomenon awaiting characterisation. The walk-frame ratio $H_{\text{walk}}/Z^* = 3/2$ accordingly reads $H/Z^* = 5/4$ for the heuristic on the same witness.

Proposition 10 (The 4/3 bound holds whenever $K^* \leq 3$)**.** *Every $b = 3$ instance with $K^* \leq 3$ satisfies $H/Z^* \leq 4/3$. Conjecture 1 is therefore open only for $K^* \geq 4$.*

Proof. For $K^* \leq 2$ the gap is zero (Corollary 3). For $K^* = 3$, Proposition 7(i) gives gap ≤ 1, and a positive gap forces $Z^* \geq 3$ (Corollary 4), so $H/Z^* = 1 + \text{gap}/Z^* \leq 1 + 1/3 = 4/3$. □

The reduction to $K^* \geq 4$ is not vacuous: the edge-family observation that a positive gap forces $K^* = 3$ does *not* extend to mixed job sizes. Positive-gap $b = 3$ instances

with $K^* = 4$ exist—for example $T = (\{0,2,5\},\{0,3,6\},\{1,2\},\{1,4\},\{1,6\})$ with $Z^* = 4$, $H = 5$ and ratio $5/4$—so an earlier draft's assumption to the contrary is false, and the $K^* \geq 4$ case is a genuine, populated regime. No ratio above $4/3$ has appeared in it. Conjecture 1 carries no load elsewhere in this report: every other statement is proved unconditionally.

Remark 4 (The extremal curve in b). The constant $4/3$ is specific to $b = 3$. The extremal ratio is non-decreasing in b: it reaches $4/3$ already at $b = 4$ and $3/2$ at $b = 5$, the latter now witnessed at $K^* = 3$ (Proposition 9) as well as at $K^* = 4$. Whether the supremum over all b stays below 2, or is bounded at all, is open; growth would require a shared bridge structure across unboundedly many groups, which the disjoint-copy family of Proposition 5 cannot provide—there the copies decouple and the ratio *decreases* to $7/6$.

3.5 What determines the gap: grouping as an independence system

The results above are quantitative; this subsection asks from which combinatorial structure of an instance the gap can, even in principle, be read. The natural object is the family of feasible groups.

Definition 6 (Group complex and circuit clutter)**.** Let $\mathcal{F} = \{S \subseteq J : |\bigcup_{j\in S} T_j| \leq b\}$. Since $|T_j| \leq b$ and membership is preserved under taking subsets, $\mathcal{F}$ is an independence system on J; its inclusion-minimal non-members, the *circuits*, form a clutter $\mathcal{C}$.

Three plain ideas hide in this definition. That $\mathcal{F}$ is an *independence system* means only that feasibility is downward closed: remove a job from a group that fits the magazine and it still fits, so "feasible group" behaves exactly like "independent set," and the standard machinery of independence systems applies. The *circuits* are the minimal ways to *fail*—a set of jobs whose tools overflow the magazine although every proper subset fits, i.e. the smallest irreducible tool-clashes. And a *clutter* is simply a family no member of which contains another (an antichain), which the circuits form automatically, since a circuit containing a smaller one would not be minimal. The question of this subsection is how much of the gap these minimal clashes already fix.

Proposition 11 (K^* is a hypergraph chromatic number)**.** *A job set is a feasible group iff it contains no circuit; hence K^* equals the chromatic number of the circuit hypergraph $(J, \mathcal{C})$. The conflict graph of Remark 3 is exactly the family of two-element circuits, so $\chi(G_{\text{conf}}) \leq K^*$, strictly in general: for $T_1 = \{0,1\}$, $T_2 = \{0,2\}$, $T_3 = \{0,3\}$ with $b = 3$ there are no conflicting pairs ($\chi = 1$), yet the triple is a circuit and $K^* = 2$.*

Proof. An infeasible set contains a minimal infeasible subset, and feasibility is downward closed, giving the first claim; colour classes of a proper colouring of $(J, \mathcal{C})$ are then exactly

feasible groups. Two-element circuits are precisely the conflicting pairs, and every proper colouring of the hypergraph properly colours the graph. The star instance computes as stated. □

In words: a proper colouring of the circuit hypergraph paints the jobs so that no minimal tool-clash is left all one colour, and each colour class is then a group that fits the magazine; the fewest colours that suffice is exactly K^*. The subtlety the proposition records is that pairwise clashes undercount—three jobs can be pairwise compatible yet jointly overflow—so the true obstruction is the hypergraph of *all* minimal clashes, not merely the two-job ones the conflict graph sees (Figure 9).

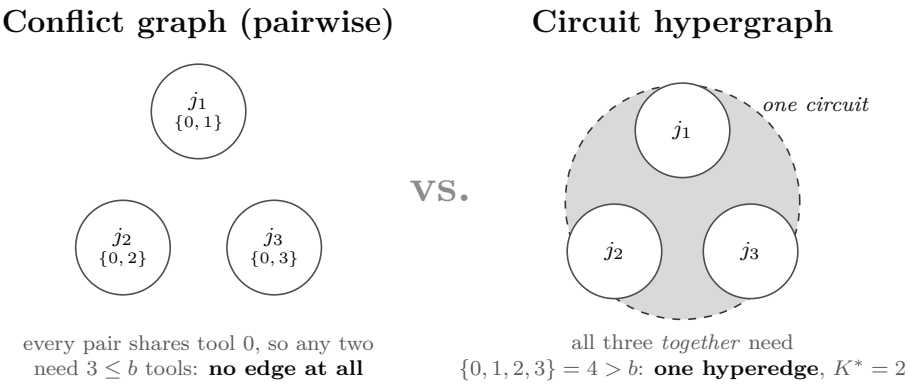

Figure 9: The circuit hypergraph, on the star instance $T_1 = \{0,1\}$, $T_2 = \{0,2\}$, $T_3 = \{0,3\}$ with $b = 3$. No two jobs conflict—each pair needs only three tools, which fit—so the conflict *graph* has no edges at all. Yet all three together need four tools and overflow the magazine: $\{j_1, j_2, j_3\}$ is a *circuit*, a minimal infeasible set, drawn here as the shaded hyperedge that no graph of pairwise conflicts can express. It already forces $K^* = 2$. This is exactly why K^* is read off the hypergraph of *all* minimal clashes (Proposition 11), never the graph of pairwise ones alone.

Proposition 12 (No matroid structure)**.** *$\mathcal{F}$ is not a matroid in general: for $T_1 = \{1\}$, $T_2 = \{2\}$, $T_3 = \{3,4\}$ with $b = 2$, both $\{3\}$ and $\{1,2\}$ are independent but neither $\{3,1\}$ nor $\{3,2\}$ is—the exchange axiom fails, so greedy and matroid-union arguments are unavailable without further hypotheses.*

Proof. Immediate: the relevant unions have sizes 2, 2, 3, 3. □

Proposition 13 (The circuit clutter does not determine the gap)**.** *Two instances with $b = 4$, $|U| = 7$, four jobs, isomorphic circuit clutters, and equal $K^* = 3$ and $Z^* = 3$ can have different gaps:*

$$I_0 : T = \big(\{0,2,6\},\{1,2,3\},\{2,3,4,5\},\{3,4,5\}\big), \qquad I_1 : T = \big(\{0,1,3,6\},\{1,4,5,6\},\{2,4\},\{3,5\}\big).$$

In both, every job pair conflicts except the last two (the pairwise unions exceed b exactly there, and no larger circuit is minimal), yet I_0 has gap 0 and I_1 has gap 1.

Proof. The pairwise-union sizes are checked directly. Exhaustively over sequences and minimum-cardinality groupings, $Z^* = 3$, $H = 3$ for I_0 and $Z^* = 3$, $H = 4$ for I_1. □

Consequently no criterion reading only the circuit clutter—a fortiori only the conflict graph, strengthening Remark 3—can decide when the gap vanishes: tool-level structure is essential. That is precisely the level at which Proposition 6 operates; its overlap sizes x_{uv} are the data the clutter forgets. On the *covering* side, however, the clutter is exactly the right object, and blocking duality makes this precise.

Proposition 14 (Circuit–group blocking identity)**.** *On ground set J, the blocker of the circuit clutter—the clutter of its minimal transversals—is the family of minimal complements of maximal feasible groups.*

Proof. A set meets every circuit iff its complement contains no circuit iff its complement is a feasible group; minimal transversals correspond to maximal feasible complements. □

This duality has a concrete payoff for the next section. Grouping branch-and-price rests on a set-covering relaxation—cover every job by feasible groups at least total cost—and it is fast only when that relaxation is *ideal*, meaning its fractional optimum is already whole-numbered, so no branching is needed. The next result exhibits the smallest family where it is not: the fractional cover slips below the integer one, exactly as a fractional colouring beats the integer one on an odd cycle.

Proposition 15 (Odd rings: a first non-ideal family)**.** *Consider the clutter of job clusters over the maximal feasible groups; its blocker is the clutter of minimal group-covers, so K^* is the blocker's minimum size. On the k-ring with $b = 3$ the fractional cover equals $k/2$ while $K^* = \lceil k/2 \rceil$: the covering relaxation is tight for every even k and has integrality gap $\frac{1}{2}$ for every odd k.*

Proof. The maximal groups are the adjacent edge pairs. Weight $\frac{1}{2}$ on each covers every job exactly once, so the fractional cover is at most $k/2$; the dual packing $y_j = \frac{1}{2}$ is feasible since no group holds more than two jobs, so by linear-programming duality it is exactly $k/2$. The integer optimum is $\lceil k/2 \rceil$. □

This locates a first provable obstruction family for the set-covering relaxation that powers grouping branch-and-price [14, 39]: odd circular structure, in exact parallel to odd holes in perfection theory. Since idealness transfers between a clutter and its blocker (Lehman's theorem), the cluster and cover sides stand or fall together; whether odd-ring-like structure is the *only* obstruction is the polyhedral question of the planned work (Section 7).

3.6 The setup-cost spectrum: from the SSP to the JGP

In practice each changeover incurs a fixed setup time beyond the per-tool exchanges; this subsection shows that the SSP and the JGP are the two ends of one parameterised problem, with the transition governed by the gap itself. Model this by charging, in addition to the tool switches, a cost $\rho \geq 0$ per *configuration change*; the objective becomes switches $+ \rho \cdot$ (#configuration changes), where ρ is the ratio of setup time to unit switch time. A grouping into p configurations makes $p - 1$ changes, so the setup term alone is minimised by the Job Grouping Problem; as ρ grows it dominates and pulls the optimum onto minimum-cardinality groupings.

Theorem 3 (Setup-cost collapse)**.** *If $\rho > H - Z^*$, then every optimal solution of the setup-augmented objective uses a minimum-cardinality grouping, and the two-phase heuristic is exact. Hence the collapse threshold satisfies $\rho_c \leq H - Z^*$; for the ring family $\rho_c = 1$.*

Proof. Every feasible grouping $\mathcal{P}$ has $|\mathcal{P}| \geq K^*$ and, by Theorem 1, $\gamma(\mathcal{P}) \geq Z^*$, so its augmented cost is $\rho(|\mathcal{P}| - 1) + \gamma(\mathcal{P}) \geq \rho(|\mathcal{P}| - 1) + Z^*$. The best minimum-cardinality grouping has augmented cost $\rho(K^* - 1) + H$. If $|\mathcal{P}| > K^*$, the former exceeds the latter whenever $\rho(|\mathcal{P}| - K^*) > H - Z^*$, which holds for all such $\mathcal{P}$ as soon as $\rho > H - Z^*$ (since $|\mathcal{P}| - K^* \geq 1$). Thus no over-cardinality grouping is optimal, and the heuristic—which selects the cheapest ordering of a minimum-cardinality grouping—attains the optimum. The ring value $\rho_c = 1$ follows from $H - Z^* = 1$ and the exact ring costs. □

Corollary 6. *On ring-like instances the setup-to-switch ratios $\rho \approx 3$–60 reported for real tooling [43, 44] far exceed ρ_c, so the two-phase heuristic is exactly optimal for the setup-augmented objective there.*

Lemma 4 (The low-ρ end of the spectrum)**.** *If $\rho < 1/(n-1)$, every optimum of the setup-augmented objective is a switch-optimal SSP schedule.*

Proof. A schedule makes at most $n - 1$ configuration changes. Let S^* be switch-optimal; any schedule with at least Z^*+1 switches has augmented cost at least $Z^*+1 > Z^*+\rho(n-1)$, which is at least the augmented cost of S^*; so no such schedule is optimal. □

Together with Theorem 3 this pins both ends of the spectrum: for $\rho < 1/(n-1)$ the augmented problem *is* the SSP, and for $\rho > H - Z^*$ its optima use minimum-cardinality groupings, so the two-phase heuristic is exactly optimal—the problem has collapsed onto the JGP side. The transition is confined to the interval $[\,1/(n-1),\ H - Z^*\,]$, i.e. it is governed by the gap itself; and since real tooling reports setup-to-switch ratios $\rho \approx 3$–60 (Corollary 6), production instances sit far on the collapsed side. This describes, for the uniform model, how the problem interpolates between the SSP and the JGP as the magazine-setup versus tool-setup ratio varies.

3.7 Which grouping to choose

Theorem 1 converts the heuristic's weakness into a concrete algorithmic question. Since the optimum is attained by *some* feasible grouping, and minimum-cardinality ones may miss it (Example 3), one should ask on *which* grouping to run the sequencing phase. Admitting a single extra group already recovers the ring optimum; in general the trade-off is between more configuration changes and cheaper individual transitions. Characterising the groupings that attain $\min_{\mathcal{P}} \gamma(\mathcal{P})$, or giving an efficient rule that improves on the minimum-cardinality choice, is open; the planned work returns to it (Section 7).

The theory thus does two things at once. It measures how far the heuristic can fall short—the gap, bounded above and below and pinned exactly at $K^* = 3$—and it locates *where* that shortfall lives: on the instances where the coverage bound $|U| - b$ is loose, the same instances on which, as the next section shows, every configuration relaxation also goes slack. There the optimum can be certified only by search. That search—and the question of whether any exact method can reach past the coverage bound the theory keeps returning to—is the subject of Section 4.

4 Exact methods

The gap analysis showed that $|U| - b$ is often the exact optimum; the question here is whether exact methods can exploit that, and what happens where the bound is loose. Two routes present themselves. Because the loading subproblem is polynomial (Proposition 1), a Benders decomposition can fix the order in a master and settle the loading exactly in the subproblem, generating cuts (Section 4.2). Alternatively the problem can be reformulated over configurations and solved by column generation—but the natural configuration formulation has exponentially many subtour constraints, and making it tractable is the obstacle that the position-indexed formulations of Section 4.4 resolve. We first record the compact formulations that serve as baselines.

Why these two methods. The choice of methods is dictated by where the difficulty sits. A compact integer formulation is the natural first recourse—and three from the literature serve here as baselines—but their linear relaxations all collapse to the coverage bound $|U| - b$ (Proposition 16), so a branch-and-cut on any of them inherits precisely the weakness the structural theory predicts: fast where that bound is tight, adrift where it is loose. Escaping it requires a method that does not rest on a compact relaxation, and two classical routes qualify, for opposite reasons. Benders decomposition is the textbook fit when the subproblem is *polynomially* solvable [45]: KTNS settles the loading exactly, so it becomes an oracle returning exact optimality cuts, and the master's bound is assembled from true subproblem values rather than from a weak relaxation—the algorithm *sidesteps*

Method	Source	Formulation	Linear relaxation
F4	Catanzaro et al. [9]	compact, arc-based	below $\lvert U\rvert - b$
LSS	Laporte et al. [29]	compact, tool-state	below $\lvert U\rvert - b$
SSPMF	da Silva and Yanasse [17]	multicommodity flow	$= \lvert U\rvert - b$
BBC	this work	branch-and-Benders-cut	none (KTNS-cut bound)
PCF, PCF′	this work	position–configuration B&P	$= \lvert U\rvert - b$
PTF	this work	position–transition B&P	$\geq \lvert U\rvert - b$, can exceed

Table 3: The three compact baselines and the exact methods of this report, ordered by the strength of their linear relaxation relative to the coverage bound $|U| - b$. Every method is exact; they differ in whether, and by what means, they get past that bound—the organising question of this section.

the coverage bound rather than trying to strengthen it. This places the solver in the lineage of logic-based [26] and combinatorial [11] Benders decomposition, whose defining trait is exactly a subproblem settled by a combinatorial oracle rather than by linear-programming duality; the combinatorial KTNS cuts of Section 4.2 are of that family, and the LP-dual cuts are the classical Benders form used alongside them. Branch-and-price attacks the same weakness from the other side: by reformulating over configurations, its position–transition variant carries a relaxation that provably *exceeds* $|U| - b$ (Proposition 19), the one construction in this report that does. The two are therefore complementary probes of a single question—whether any exact method can reach past the coverage bound—one by never forming the weak relaxation, the other by improving on it.

The optimization toolkit, in brief. The methods in this section draw on a small, standard vocabulary, gathered here so that nothing that follows is left undefined; a reader fluent in integer programming may skip to Section 4.1. An *integer linear program* minimises a linear function of decision variables under linear constraints, with some variables forced to whole numbers—here almost always 0/1 variables encoding discrete choices, such as "job j runs immediately after job i" or "tool t occupies the magazine at position i." The integrality is the whole of the difficulty: dropping it leaves the *linear relaxation*, an ordinary linear program that solves quickly and whose optimum is a *lower bound* on the true minimum. The distance between the two is the *integrality gap*, where essentially all the hardness lives; a formulation is called *strong* when that gap is small, and much of this section is a contest over which formulation bounds the optimum Z^* most tightly.

Branch-and-bound turns such a bound into a proof of optimality by search: it solves the relaxation, and when a variable comes out fractional it splits the problem in two—forcing that variable up or down—recursing into a tree whose branches are pruned the instant their relaxation bound is no better than the best schedule already found. *Cutting planes* sharpen this by adding valid inequalities ("cuts") that remove a fractional relaxation solution without deleting any integer one, lifting the bound before the search branches;

the combination, *branch-and-cut*, is the engine inside modern solvers such as the CPLEX system on which every method here is built.

Two structural ideas complete the kit. *Linear-programming duality* pairs each linear program with a second one of equal optimum whose variables *price* the constraints; those prices are exactly what hand the Benders algorithm its cuts at no extra cost (Section 4.2). And *decomposition* exploits problems that split into a hard part and an easier one: *Benders decomposition* fixes the hard part and lets the easy subproblem generate cuts (Section 4.2), whereas *column generation* runs the same idea in reverse, letting a subproblem invent new variables for a master too large to write down (Section 4.4). Both trade one intractable model for a sequence of small, solvable ones—the move this section makes again and again.

4.1 Compact formulations and the coverage relaxation

Three compact formulations from the literature serve as the baselines of the computational study and as the yardstick against which relaxation strength is measured. Catanzaro et al. [9] give a family F1–F5 in job–position and ordering variables, the tighter members adding linear-ordering and overlap inequalities; we use F4 as the baseline because it is the strongest member that scales: its linear relaxation coincides with F3's at a smaller model size, while F5's tighter relaxation is so large that at the $n = 40$ benchmark sizes even its root linear relaxation does not complete. Laporte et al. [29] formulate the problem with tool-state variables and a branch-and-cut. da Silva and Yanasse [17] give a multicommodity-flow model in which each tool routes one unit of flow through the position network. The last is singled out by its relaxation.

Proposition 16. *The linear relaxation of the multicommodity-flow formulation equals the coverage bound $|U| - b$ of Proposition 3.*

Established by da Silva and Yanasse [17], and matched exactly by the position formulation's relaxation with counting rows (Proposition 18), this matters twice: the compact relaxation is no stronger than the elementary counting bound; and since $|U| - b$ equals Z^* on many families (Section 3), the model is fast exactly where that bound is tight and weak where it is not—a dichotomy quantified in Section 5.3.

4.2 A branch-and-Benders-cut algorithm

Idea. A schedule is an order together with a loading. Once the order is fixed, the minimum loading cost is computed exactly and in polynomial time by KTNS; so we place the *order* in an integer master and represent the loading cost by a single continuous variable θ, refined by cuts generated from the exactly solved loading subproblem. In

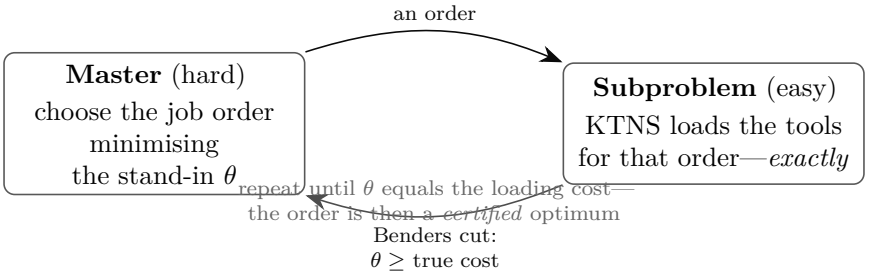

Figure 10: The branch-and-Benders-cut loop. The master picks an order using an optimistic stand-in θ for the loading cost; the KTNS oracle returns the *exact* loading cost of that order; if θ undershot, one cut lifts it, and the cut is valid for every order at once. Because the subproblem is polynomial each cut is exact, and the loop halts at a proven optimum. The cuts are separated inside a branch-and-bound tree over the order variables—hence *branch*-and-Benders-cut.

contrast to column generation, whose pricing subproblem for this problem is NP-hard (Section 2.7), the Benders subproblem here is polynomial, so every cut is exact.

What Benders decomposition does. The manoeuvre is worth stating without notation, because it recurs throughout this section. Suppose a problem splits into a hard part and, once the hard part is fixed, an easy part—here, choose the order (hard), then load the tools (easy). Benders decomposition solves the hard part on its own, but with the easy part's cost stood in for by a single variable θ that begins optimistically low. Given a candidate order, it solves the easy loading problem exactly and checks whether θ told the truth. If the real loading costs more, it adds one linear inequality—a *cut*—asserting, in effect, "any order resembling this one must admit at least this much loading cost," and re-solves. Each cut deletes the current over-optimistic guess without deleting a single genuine schedule, so θ is driven upward until it equals the true loading cost, at which point the order it certifies is optimal (Figure 10). What makes the scheme both exact and fast here is that the easy part really is easy: KTNS solves it in polynomial time and its linear-programming dual yields the cut at no extra cost. This is the decisive contrast with column generation, whose analogous subproblem for the SSP is itself NP-hard.

Master. Introduce a depot 0 and arc variables $x_{ij} \in \{0,1\}$ for $i,j \in \{0,1,\ldots,n\}$, $i \neq j$, with $x_{ij} = 1$ when job j is processed immediately after i (the depot marking start and end). The assignment constraints

$$\sum_{i \neq j} x_{ij} = 1 \quad (\forall j), \qquad \sum_{j \neq i} x_{ij} = 1 \quad (\forall i),$$

together with subtour-elimination constraints $\sum_{i,j\in S} x_{ij} \le |S| - 1$ for $S \subseteq \{1,\dots,n\}$, make the x a Hamiltonian order of the jobs. The master is

$$\min\ \theta \qquad \text{s.t.} \qquad \theta \ge \textstyle\sum_{i,j} w_{ij}\, x_{ij}, \quad \text{assignment, subtour, and Benders cuts,}$$

in which θ lower-bounds the loading cost and the subtour and Benders constraints are separated lazily. The subtour separation is exact by design: violated constraints are detected at integer candidate solutions by connected-component search, so no optimal solution is ever cut off; a fractional separation routine would be an acceleration, not a correctness requirement, and is deliberately absent from the measured configuration.

An initial valid inequality. At the boundary between consecutive jobs i and j at least $w_{ij} := \max\bigl(0, |T_i \cup T_j| - b\bigr)$ tools are switched: the magazine holds at most b tools yet must serve first T_i and then T_j, so the members of $T_i \cup T_j$ absent at position i—at least $|T_i \cup T_j| - b$ of them—are inserted by position j. Summing over the chosen arcs gives the valid inequality $\theta \ge \sum_{ij} w_{ij} x_{ij}$, used to initialise the master.

Subproblem. Fix the order and relabel the jobs $1,\dots,n$ along it. With $g_{t,i} \in [0,1]$ indicating that tool t occupies the magazine at position i and $s_{t,i} \ge 0$ its insertion there, the loading cost is the optimal value of

$$\begin{aligned}
\min\ & \sum_{t,i} s_{t,i} && \\
\text{s.t. } & g_{t,i} = 1 && (t \in T_i),\\
& \textstyle\sum_t g_{t,i} \le b && (\forall i),\\
& s_{t,i} \ge g_{t,i} - g_{t,i-1}, \quad s_{t,i} \ge 0 && (\forall t, i),
\end{aligned}$$

with $g_{t,0} = 0$. The constraint matrix has the consecutive-ones property and is totally unimodular, so the relaxation is integral and, by Proposition 1, its optimum equals KTNS of the order. Writing R_i for the tool set of the job at position i, an optimal dual prices each requirement "tool t present at position i" by a multiplier $\bar\alpha_{t,i}$ and each capacity bound by $\bar\beta_i \ge 0$; linear-programming duality then yields the Benders optimality cut

$$\theta \ \ge\ \sum_i \sum_{t\in R_i} \bar\alpha_{t,i} \ -\ b \sum_i \bar\beta_i$$

(the multipliers of the unit upper bounds on the $g_{t,i}$ contribute a further term, suppressed here). The requirement multipliers carry the dependence on the order—through which job, hence which R_i, occupies each position—so the right-hand side is linear in the master's assignment variables; and because the dual feasible region does not depend on the order, the cut is globally valid, the defining property of a Benders cut.

Cut families. Two kinds of cut are separated, and the computational study switches each on and off independently to measure its contribution. *(i) Benders optimality cuts* from the dual above, separated at integer master solutions and, optionally, at fractional ones (*fractional cuts*) to tighten the bound before branching. *(ii) Combinatorial KTNS cuts*: at an integer order $\bar{\sigma}$ with $\mathrm{KTNS}(\bar{\sigma}) = \bar{z}$, the inequality

$$\theta \;\geq\; \bar{z}\Big(1 - \sum_{(i,j)\in A(\bar{\sigma})} (1 - x_{ij})\Big)$$

is valid—it forces $\theta \geq \bar{z}$ when the order is exactly $\bar{\sigma}$ (all arcs $A(\bar{\sigma})$ selected) and is vacuous otherwise—and uses only the polynomial oracle, with no dual computation. Finally, *triplet bounds* generalise w_{ij} to ordered triples of jobs, contributing $O(n^3)$ valid inequalities that strengthen the root relaxation.

4.3 Accelerating the branch-and-Benders-cut solver

The competitiveness assessment of Section 5.3 locates the solver's difficulty precisely: on the instances it fails, the incumbent already equals the optimum and only the *dual* bound lags, so the algorithm is bound-limited rather than search-limited. Two levers address this, and both are classical elements of the mature toolbox for accelerating Benders decomposition surveyed by Rahmaniani et al. [45]: strengthen the cuts, so the dual bound climbs in fewer iterations, and seed the search with a strong primal bound and an initial pool of cuts, so it prunes from the root. We instantiate four such elements. Each is an independent switch in the solver, and each was checked to preserve optimality against brute force on small instances before any campaign run, so that a strengthening can never silently return a wrong optimum.

Fractional Benders cuts. The optimality cut of Section 4.2 is derived at an integer order, where the loading dual is exact; nothing, however, prevents separating the same cut at the *fractional* solutions the linear relaxation produces at a node, before branching. Doing so attacks *tailing-off*—the long tail of nodes over which a pure integer-cut scheme inches the bound upward—by cutting the relaxation where it is loosest [45]. The intuition is that an integer-only scheme learns about the loading cost only once an order is fully committed, whereas a fractional cut extracts the same dual information from a partial, fractional order and removes it from the relaxation immediately. In this implementation the fractional separation had, until recently, been inert: every cut it constructed was discarded by an incorrect solver call and the campaign therefore recorded none. With the call corrected the cuts enter the master, and the question the re-run settles is whether the dual bound they produce rises above the coverage value $|U| - b$ on the bound-loose instances—precisely where the compact relaxations cannot reach.

A strong primal at the root. A branch-and-bound search prunes in proportion to the quality of its incumbent, and the strongest known heuristic for the SSP is the hybrid genetic search of Mecler et al. [33]. We run a compact form of it once, at the root: a population of constructive seeds—nearest-neighbour orders on the pairwise weights w_{ij} and a largest-tool-first order—each improved by a variable-neighbourhood local search (adjacent swaps and short segment relocations, every candidate scored by the exact KTNS oracle), evolved under order crossover with a diversity guard. The resulting order is used twice. As a *warm start* it hands CPLEX an incumbent from node zero, so a bound-tight instance that would otherwise be found only after search is pruned at once; and it *seeds a Benders cut*, the loading dual being solved once for the heuristic order and its optimality cut added before the first relaxation—an instance of the initial-cut-pool acceleration of Rahmaniani et al. [45]. The intuition is that a good order is not merely an upper bound but a certificate of one face of the loading value function; seeding its cut hands the master that face for free.

A conflict-graph bound. The master's root bound is pairwise plus coverage, and both are visible to the linear relaxation; the combinatorial structure they miss lives, as the structural analysis of Section 3 shows, in the conflict graph G, whose vertices are the jobs and whose edges join pairs that cannot share a configuration ($|T_i \cup T_j| > b$). A clique of G is a set of pairwise-incompatible jobs, which must occupy that many distinct configurations, so $K^* \geq \omega(G)$ and, by the grouping bound, $Z^* \geq \omega(G) - 1$; more generally $K^* \geq \chi(G)$, and where G is non-bipartite—an odd hole—one has $\chi(G) \geq 3$ even when $\omega(G) = 2$. We add the resulting constant row $\theta \geq (\chi_{\mathrm{lb}} - 1) + \min(b, |U|)$ to the master whenever it strictly exceeds the coverage bound. This is the elementary end of the conflict-graph machinery of Atamtürk et al. [3], and it is honest to note its reach is narrow: on most instances the coverage bound already dominates it, and it bites only where the grouping structure, not the tool count, is what forces switches. It is kept because it is exactly the lever the structural analysis of Section 3 predicts, and it costs a single row.

Pareto-optimal cut lifting. When the loading dual has several optimal solutions—which, being a degenerate transportation-type problem, it routinely does—the Benders cuts they yield are not equally strong: some are dominated, removing less of the relaxation than others valid at the same point. Selecting the deepest, non-dominated cut is the Pareto-optimal cut of Magnanti and Wong [32]: among the alternative dual optima, choose the one maximising the cut's value at a fixed interior "core" point of the master, which aims the cut at the interior rather than the boundary and so removes more of the relaxation. Papadakos [40] makes this practical by observing that the core point need not be tied to the current relaxation and may simply be updated as a convex combination of the points visited, sparing the extra constrained solve of the original method. We

follow the Papadakos form: a core point x^0 is kept in the relative interior of the master polytope (initialised from the heuristic order blended with the barycentre), and at each fractional separation the loading dual is solved a second time *at the core point*, its cut added alongside the ordinary one, after which x^0 is drawn halfway toward the current relaxation point. Adding the core-point cut *in addition to*—never instead of—the ordinary cut keeps the scheme valid by construction: the ordinary cut already separates the point and secures convergence, while the Pareto cut only deepens the relaxation. The intuition is that the ordinary cut is aimed at the fractional point in front of it, whereas the Pareto cut is aimed at the centre of the feasible region, so it is useful not only at this node but at the many still to come.

Each of the four is an independent option in the campaign harness, so the study of Section 5 measures their separate and combined effect against the corrected Benders base. Whether, together, they carry the solver's frontier into the bound-loose half—or merely confirm that the frontier is the bound itself—is the question the accelerated campaign is designed to answer.

4.4 Position-indexed formulations

The idea, in words. The configuration walk of Section 2.4 is the most faithful exact picture of the SSP, but as a model it has two flaws: there are far too many configurations to enumerate, and forbidding the "subtours" that a valid walk must avoid takes exponentially many constraints. The position-indexed formulations remove both flaws by fixing the schedule's length in advance. In place of an open-ended walk they lay out a fixed row of *positions*—slots $1, 2, \ldots$ into which configurations are placed—and ask, of each position, which configuration sits there and what it costs to change from the previous one. Numbering the positions does two useful things at once: it makes the transition cost a purely local quantity between adjacent slots, so the awkward global subtour constraints vanish, and it caps the model at a polynomial number of rows. The price is symmetry—the same schedule can fill the positions in several equivalent ways—which the refined formulation PCF′ then factors out. Two variants follow: the *position–configuration* formulation PCF, which names the configuration at each position directly, and the *position–transition* formulation PTF, which instead records the *change* between consecutive positions and, as the computational study shows, can lift its relaxation above the coverage bound $|U| - b$ that traps every compact model—the one route seen here to a bound stronger than the elementary one.

From the configuration model to a polynomial row set. The configuration formulation of Section 2.4, formalised by Colares and Wagler [12], is the natural exact model over configurations. As first stated, however, its subtour elimination is invalid; the

correction of Moreira [36], through prize-collecting travelling-salesman constraints, restores exactness at the price of *exponentially many* constraints, one per subset of configurations. A Miller–Tucker–Zemlin reformulation [18, 34] is the standard device for removing such constraints, but here it trades one obstacle for another.

Proposition 17. *A per-configuration MTZ potential is exact but leaves one potential per configuration, hence exponentially many; aggregating the potentials to one per job is polynomial but no longer exact—it can exclude the optimum.*

The aggregated form fails under job domination: if $T_i \subseteq T_j$, every configuration serving j also serves i, so a transition between two such configurations places $\{i, j\}$ at both ends and forces the potentials of i and j into a two-cycle no ordering can satisfy; a four-job instance with $b = 3$ has optimum 1 that the aggregated model values at 2. Neither MTZ variant yields a tractable exact model. The position-indexed formulations below take a different route: they fix, in advance, a row set polynomial in $(n, |T|)$, so column generation only ever adds columns to existing rows, and they carry no subtour constraints at all.

Position–configuration formulation (PCF). Let $y_C^k \in \{0, 1\}$ indicate that configuration C occupies position k ($k = 1, \dots, n$) and let $w_t^k \geq 0$ count the insertion of tool t at position k:

$$
\begin{aligned}
\min \quad & \sum_{t,k} w_t^k \\
\text{(P)} \quad & \sum_{C} y_C^k = 1, \quad k = 1, \dots, n; \\
\text{(Cov)} \quad & \sum_{k} \sum_{C \supseteq T_j} y_C^k \geq 1, \quad j \in J; \\
\text{(W)} \quad & w_t^k \geq \sum_{C \ni t} \left(y_C^k - y_C^{k-1} \right), \quad t \in T,\ k \geq 2; \\
\text{(T)} \quad & \sum_{k \geq 2} w_t^k + \sum_{C \ni t} y_C^1 \geq 1, \quad t \in U.
\end{aligned}
$$

The configuration variables y are generated as columns; the rows (P), (Cov), (W), (T) number $O(n|T|)$ and never change. Constraint (P) places one configuration per position, (Cov) serves every job, and (W) charges an insertion of t whenever it enters the magazine.

Proposition 18. *PCF is exact: its integer feasible points are exactly the SSP schedules, of equal cost. Its linear relaxation* without *(T) equals* 0*;* with *the counting rows (T) it equals* $|U| - b$.

The vanishing of the plain relaxation is the fractional pathology of Tang and Denardo [47]: a single convex blend of configurations, repeated at every position, satisfies (P), (Cov) and (W) at zero cost. The counting rows (T)—each tool of U is inserted at least once unless present at position one—restore the coverage bound, matching the multicommodity model (Proposition 16) with one extra family of rows.

Position–transition formulation (PTF). To exceed the coverage bound we index *transitions* rather than positions. Augment the configurations with an absorbing tool-less node $\perp$, used to pad schedules that change configuration fewer than $n-1$ times, and let $z^k_{C,C'} \geq 0$ be a column for the step $k \to k{+}1$ that leaves configuration C and enters C'. The formulation is

$$\begin{aligned}
&\min \sum_{k=1}^{n-1} \sum_{(C,C')} d(C,C')\, z^k_{C,C'} \\
&\text{(S)} \quad \sum_{(C,C')} z^k_{C,C'} = 1, \qquad k = 1,\dots,n-1; \\
&\text{(F)} \quad \sum_{(C,C')\colon t \in C'} z^k_{C,C'} = \sum_{(D,D')\colon t \in D} z^{k+1}_{D,D'}, \qquad t \in T,\ k = 1,\dots,n-2; \\
&\text{(Cov)} \quad \sum_{k=1}^{n-1} \sum_{(C,C')\colon T_j \subseteq C} z^k_{C,C'} \;+ \sum_{(C,C')\colon T_j \subseteq C'} z^{n-1}_{C,C'} \;\geq\; 1, \qquad j \in J.
\end{aligned}$$

Constraint (S) selects one transition per step; the flow-consistency rows (F) require, tool by tool, that what enters position $k{+}1$ at the end of step k is what leaves it at the start of step $k{+}1$, so the columns describe a single coherent walk $C^1 \to \cdots \to C^n$; and (Cov) serves every job at the configuration of some position—the tail of a step, or the head of the last. The objective charges $d(C,C') = |C' \setminus C|$ per transition. As in PCF the rows number $O(n|T|)$ and are fixed, while the transition columns are generated.

Proposition 19. *PTF is exact, and its linear relaxation is at least $|U| - b$ and can be strictly greater.*

On an instance whose optimum exceeds both bounds of Corollary 1, the PTF relaxation returns 2.10 against a coverage bound of 2; on the 6-ring it is tight at $Z^* = 3$. PTF is thus the first relaxation in this family to improve on the elementary bound, and it does so with a fixed, polynomial row set.

Symmetry reduction: the formulation PCF′. PCF carries a large *padding* symmetry. A schedule using $m < n$ distinct configurations is encoded by repeating configurations to fill all n positions, and the repeats may sit anywhere, so one physical schedule has many equal-cost encodings—distinct nodes to a tree search. We remove this freedom by relaxing the assignment equation (P) to an inequality and forcing the empty positions to a suffix:

$$\text{(P}') \quad \sum_C y^k_C \leq 1 \ \ (\forall k); \qquad \text{(G)} \quad \sum_C y^{k+1}_C \leq \sum_C y^k_C \ \ (k = 1,\dots,n-1);$$

with (Cov), (W), (T) unchanged. Call this formulation PCF′. At an integer point the counts $\sum_C y^k_C$ are non-increasing in k, so the *used* positions form a prefix $1,\dots,p$ and the empties a suffix, with the cost-free first configuration at position 1. The contiguity rows

(G) are not decorative: without them an interior empty position would make (W) read the re-entry as a full reload and (T) misattribute the free load, so (G) is what keeps the relaxed model faithful.

Proposition 20. *PCF′ is exact, and its linear relaxation equals that of PCF—$|U| - b$ with the counting rows (T), and* 0 *without. It removes symmetric integer encodings but does not strengthen the bound.*

The proof is in Appendix A. The separation of concerns is deliberate: PCF′ attacks symmetry, and only PTF attacks the bound.

What column generation does. The position-indexed models carry one variable for each (configuration, position) pair, and there are astronomically many configurations, so the models cannot be written out in full. Column generation is the standard escape. It keeps only a handful of configurations active, solves the resulting small linear program, and then poses a separate *pricing* problem: is there any configuration, currently omitted, that would improve the solution if it were added? The pricing problem searches configuration space using the current linear-programming prices, returns the most promising configuration—a new "column"—and the small model is re-solved with it included; the loop stops when no improving configuration exists, at which point the restricted model is provably as strong as the full one. Wrapping this inside branch-and-bound gives *branch-and-price*, an exact method—provided branching does not distort the pricing problem, which is why the branching here is arranged not to touch the pricer at all: it fixes whether a tool occupies a position (the robust presence-variable rule developed below), never an individual column. The two formulations below differ precisely in how hard their pricing problem is, and that, rather than the size of the master, is what decides whether the approach is practical.

Column generation and robust branching. Both PCF′ and PTF have $O(n|T|)$ rows but exponentially many columns ($\binom{|T|}{b}$ configurations, or arcs), so their relaxations are solved by *column generation*: a restricted master over a small column pool is reoptimised, and a pricing subproblem repeatedly supplies a column of most-negative reduced cost until none remains. Driving this at every node of a branch-and-bound tree gives *branch-and-price* [4, 31]. Branching on a single column y^k_C is ineffective under the position symmetry and forces the pricer to carry a growing list of forbidden columns. We branch instead on the *presence* variables $a^k_t = \sum_{C \ni t} y^k_C \in \{0, 1\}$ already appearing in (W): fixing a^k_t to 0 or 1 forbids or forces tool t at position k, which in the pricing subproblem merely shifts one dual weight and leaves the oracle's *form* unchanged. This *robust* branching keeps pricing and branching compatible at every node.

Pricing PCF′: a b-subset with coverage bonuses. Let $\alpha_k, \gamma_k \le 0$, $\pi_j, \mu_{t,k}, \lambda_t \ge 0$ be the duals of (P′), (G), (Cov), (W), (T). Direct dual accounting (collecting the coefficient

of y_C^k, which enters (W) and (T) through $a_t^k = \sum_{C \ni t} y_C^k$) gives the reduced cost

$$\bar{c}(C,k) = \beta_k - \sum_{t \in C} \rho_t^k - \sum_{j:\, T_j \subseteq C} \pi_j, \qquad \rho_t^k = -\mu_{t,k}[k \geq 2] + \mu_{t,k+1}[k \leq n-1] + \lambda_t\,[k = 1,\ t \in U],$$

with β_k depending only on the position. For each k the most-negative reduced cost is obtained by maximising the configuration-dependent part,

$$\max_{|C|=b} \sum_{t \in C} \rho_t^k + \sum_{j:\, T_j \subseteq C} \pi_j,$$

i.e. choosing b tools to collect per-tool rewards plus a bonus for each job whose *entire* requirement lands inside C. This is a Set-Union Knapsack Problem [23]—the grouping-pricing class of Crama and Oerlemans [14]—solved by enumerating the $\binom{|T|}{b}$ subsets at magazine sizes, or by a compact mixed-integer program. The branching dual of a_t^k enters additively in ρ_t^k, so branching does not change the oracle.

Pricing PTF: a coupled pair of subsets. For PTF a column is an arc (C, C'), and its reduced cost contains the bilinear term $\sum_t [t \in C' \setminus C]\,(1 - \lambda_t)$ together with chaining-dual terms on the tools of C and C'. The pricer cannot therefore separate into a single set: it must choose *two* b-subsets $C \neq C'$ at once, a bilinear selection linearised by McCormick on the products $x'_t(1 - x_t)$ and solved as a mixed-integer program with $2|T|$ binaries. This heavier pricing is the algorithmic price of PTF's stronger bound. The contrast—PCF′ pairing a weak bound with a cheap single-set pricer, PTF a stronger bound with a costly coupled-pair pricer—is the central trade-off assessed in Section 5.3.

Accelerating the branch-and-price. As with the Benders solver, the prototype above is deliberately left unaccelerated, so that its solve counts measure the *formulation*—the strength of the position relaxation—and not the engineering around it. Four standard accelerations, the column-generation counterparts of the Benders levers of Section 4.3, are nonetheless what a competitive implementation needs, and each is an independent switch in the campaign harness. *Heuristic pricing* answers the pricing question first with a fast greedy b-subset and calls the exact mixed-integer oracle only to *certify* that no improving column remains; exactness is untouched—termination still rests on the exact oracle—while the costly oracle, the measured bottleneck at scale, is skipped on all but the final round. *Multiple pricing* returns several negative-reduced-cost columns per round instead of one, cutting the number of master reoptimisations. *Warm starting* seeds the initial column pool from a good heuristic schedule—a grouping or greedy solution—in place of the trivial one-configuration-per-job pool, so the master begins near the optimal face rather than far from it. *Dual stabilisation* [31] smooths successive dual vectors to damp the oscillation that slows column generation on the degenerate masters these formulations produce. None

of the four alters the relaxation value or the optimum—they are speed, not strength—so, exactly as for the Benders accelerations, the campaign switches each on independently and reads off its separate and combined contribution to the solve rate.

4.5 Learning to select cuts: a reinforcement-learning prototype

The branch-and-Benders-cut solver of Section 4.2 produces, at a given node, *several* admissible cuts: a combinatorial KTNS cut for each candidate order it evaluates, the coverage inequality, and—once fractional separation is enabled—one cut per violated fractional point. Which of these to add, and in what order, is a decision the solver now makes by a fixed rule, and it is the natural place to ask whether a *learned* policy would do better. This subsection records a prototype in that direction, built alongside the deterministic solver and following the cut-selection framework of Tang et al. [48]; its evaluation on the SSP awaits the campaign, for the reason given at the end.

The cut-selection Markov decision process. Following Tang et al. [48], cut selection is cast as a Markov decision process. At separation round t the state s_t is the current relaxation solution together with a pool of candidate cuts, each summarised by a feature vector ϕ that collects the cut's *violation* at the current point, its *support size* (density), its right-hand side and its mean coefficient magnitude. An action a_t selects one candidate to add; the reward r_t is the resulting increase in the dual bound—how much that cut tightened the relaxation. The policy is a softmax over the candidates, linear in the features,

$$\pi_\theta(a \mid s) = \frac{\exp\!\big(\theta^\top \phi_a\big)}{\sum_{a'} \exp\!\big(\theta^\top \phi_{a'}\big)},$$

and its parameters θ are trained by the REINFORCE policy-gradient rule with a return baseline,

$$\theta \;\leftarrow\; \theta + \alpha \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\,\big(G_t - \bar{G}\big), \qquad G_t = \sum_{k \ge t} \gamma^{\,k-t} r_k,$$

so that a cut whose selection is followed by a large bound improvement is made more likely. A softmax-linear policy keeps the learned rule interpretable—the sign and size of each component of θ report how much each feature matters—while remaining expressive enough to separate useful cuts from useless ones.

What was built and verified. Because the branch-and-Benders-cut solver runs on a commercial mixed-integer solver unavailable in the development environment, the learning machinery was implemented and validated on an exactly-valid, self-contained cut family—knapsack cover cuts—whose separation and bound effect can be computed directly. On

held-out instances the trained policy raises the linear-relaxation bound about 60% more per episode than uniform-random cut selection, and it does so by discovering, unaided, that a cut's *violation* is the dominant feature—exactly the quantity a hand-written rule would privilege. The learned policy and its feature map are exposed through a scoring function that ranks candidate Benders and combinatorial cuts inside the solver's callback, so that carrying the prototype over to the SSP is a matter of feeding it the same features on real cuts.

Why it is reported, but gated. The honest limit is the one drawn in Section 5.3. The frontier that defeats the exact solver is a *bound* gap: on the hard instances the incumbent already equals the optimum and only the dual bound lags, so where no strong cut exists a learned selector has nothing to select that would close the instance. Cut selection can reorder and hasten the cuts that *do* exist—worth having—but it cannot manufacture bound strength. The prototype is therefore presented as a capability with a verified learning core and a ready integration path, to be evaluated on the SSP only once the campaign identifies the bound-loose instances on which it could plausibly help; it is not offered as a substitute for the deterministic baseline this report measures.

5 Computational study

Computation in this project is the measuring instrument of the theory: the quantity the theorems single out, the coverage bound $|U| - b$, is also the root relaxation of every configuration method, so a benchmark that records optima and root bounds measures the theory's central variable across the standard instance suites. The campaign also answers a question that predates the theory and motivated the solver in the first place: the SSP has a polynomial loading subproblem—the textbook precondition for Benders decomposition—and whether a decomposition can exploit it against modern compact models had never been tested. The methods of Section 4 were implemented and benchmarked; this section fixes the experimental design and states the questions the definitive campaign settles—foremost whether the decomposition, now that its fractional cuts are corrected and its root accelerations enabled, lifts the dual bound past the coverage value on the instances where it is loose. The preliminary runs reported below indicate the decomposition solves as far as its root bound reaches; the campaign under the final protocol (Section 5.1) is in progress.

5.1 Experimental design

The benchmark draws on the standard SSP instance families of Catanzaro et al. [9] (n = 8–40 jobs, m = 5–60 tools, b = 3–30), Crama et al. [15] (n = 10–40, m = 10–

60, $b = 4$–30) and Laporte et al. [29] ($n = 8$–15, $m = 10$–25, $b = 4$–20), spanning a wide range of sizes, tool densities and capacities. Three things are measured. *(i)* The branch-and-Benders-cut solver is run in an ablation over its cut families—the LP-dual and combinatorial strategies, each with and without fractional separation—together with each of its three root accelerations (a conflict-graph bound, a hybrid-genetic-search warm start, and Papadakos cut lifting), against the three compact baselines. *(ii)* The position-indexed branch-and-price is run in its PCF′ and PTF variants, on the instances with at most 25 jobs—the prototype's validated range; the compact solvers and the Benders algorithm face the full sets. *(iii)* For every solver and instance the *root linear-relaxation value* is recorded alongside the optimum where the solver exposes it reliably, so that bound tightness can be studied directly. The common metric is the empty-start KTNS cost of the returned sequence, well defined for every method regardless of its native objective. All runs were executed on the LIMOS SLURM cluster, confined to a homogeneous pool of identical nodes (AMD EPYC 7452, 2×32 cores, 512 GB RAM), so that wall-clock times are comparable across runs. The mixed-integer solver is CPLEX 22.1.1 throughout, pinned to four dedicated threads per run; the branch-and-price prototypes run single-threaded under PySCIPOpt. Each run is allotted 16 GB of memory and a single, uniform per-instance time limit of one hour (3600 s) across all families; no instance is skipped, so the number solved is reported against the full family size rather than an outcome-dependent "attempted" count. All other solver parameters are left at their defaults, so that the comparison measures the formulations rather than parameter tuning. The complete implementation—the solvers, the reimplemented baselines, the campaign harness, and the per-instance result files behind every number in this section—is available at `https://github.com/shankar-n/JGP-SSP-Codes-Shankar`.

Reporting. Results are reported per instance family, with a performance profile—the number of instances each method solves within a given time budget—and shifted-geometric-mean solve times on the instances a set of methods commonly solves, following standard practice for exact-method comparisons.

Status: what these numbers are, and are not. Candour is owed about the provenance of the figures in this section, because two things have changed since they were produced, and both are corrected in the campaign now running. First, they predate the correction of the fractional-cut injection defect described in Section 4.3: in the run reported here the fractional-cut configurations never actually added a cut, so their columns measure the base Benders solver rather than fractional Benders cuts, and *no* conclusion about those cuts should be drawn from them. Second, the numbers were collected under a provisional two-tier protocol that split the suites into a *primary* set solved to a 3600-second limit and a *secondary* set to a 600-second limit, with an early-stop rule that retired a

configuration after a run of consecutive timeouts—which is why several cells below carry outcome-dependent denominators and dashes instead of the full instance count. That protocol has since been replaced (Section 5.1) by the single uniform one used from now on: every instance of every family, one 3600-second limit, nothing skipped, reported per family with a performance profile. The corrected, uniform campaign is executing on the cluster as this interim report is submitted, and its results will replace Table 4 in the final version. The numbers below are therefore *preliminary*, and are kept for one purpose only: to fix the questions the campaign settles and to state, in advance, what the theory predicts their answers will be.

Validation. Before the campaign, all four exact solvers were checked to reproduce brute-force optima on a bank of small instances (both cut modes of the Benders solver included), and the harness converts every native objective to the common empty-start convention before any comparison. During the campaign the check is global: across the 806 instances solved by at least one method, no two methods ever disagree on the canonical cost.

Scope and limitations. The branch-and-price prototypes are deliberately *unaccelerated*: no column-pool warm start, no dual stabilisation, no heuristic pricing; and branching is the robust scheme on the aggregated variables $a_{t,p}$, which leaves the pricing problem structurally intact (the Ryan–Foster rule solves a problem this master does not have—it exists for set-partitioning masters whose pricers variable branching would destroy). The reason is that the prototype serves here as a measuring instrument for the relaxation itself, and every acceleration would confound the measurement. The results below justify the choice and delimit it: they separate the failures a faster implementation would convert (instances whose optimum already equals the root bound) from the failures no implementation can convert (instances where the bound itself falls short). The engineering agenda—warm columns from the constructive heuristics, heuristic pricing with exact certification, stabilisation—is planned work aimed at exactly the first class.

5.2 Results

Table 4 reports, per method and per instance family, the number of instances solved to proven optimality over the number attempted under the early-stop protocol of Section 5.1. Reading guide: denominators are outcomes (a retired configuration stops attempting); a dash means the protocol retired the configuration before it reached that family; and the counts are mutually consistent in the sense of the validation paragraph above.

Compact models. The multicommodity-flow model is the strongest method on the primary families in both dimensions, replicating its state-of-the-art claim [17]. On the 91

Suite	Family (n; m)	LSS	CATZ-F4	SSPMF	BBC-LP	PCF′	PTF
primary	Catanzaro A (8–10; 5–10)	51/51	51/51	51/51	24/45	1/1	1/1
	Catanzaro B (15; 18–20)	0/7	0/8	21/36	—	—	—
	Catanzaro C, D (30–40)	—	—	—	—	0/48	0/48
	Crama (10–40)	—	—	—	—	0/48	0/48
	Laporte T7 (10–15)	40/40	40/40	66/78	19/28	—	—
secondary	Laporte T3 (8)	339/339	340/340	n/a	244/318	91/302	71/288
	Laporte T4 (9)	328/330	115/150	n/a	—	2/16	0/2
	Laporte T5 (15)	0/8	—	n/a	—	—	—
totals	primary, of 411	91	91	138	43	1	1
	secondary, of 1,010	667	455	n/a	244	93	71

Table 4: Solved/attempted per method and family under the early-stop protocol; the totals rows use *uniform* denominators (the whole suite), counting every unattempted instance as unsolved, so methods are directly comparable. "—": the configuration was retired before reaching the family; "n/a": not run by design (Section 5.1). Time limits 3600 s (primary) and 600 s (secondary). BBC-LP is shown for the Benders solver *with the corrected master* (coverage row included); after the correction all eight Benders configurations perform within a narrow band (the combinatorial-cut family solves 43/73 primary and 239/318 secondary), whereas before it the best configuration solved 1/9 and 26/48. The branch-and-price columns cover the validated range $n \leq 25$ plus a separate $n \geq 30$ extension run (the Catanzaro C/D and Crama rows), where both prototypes solve nothing.

primary instances that all three compact models solve, its shifted geometric-mean time is 2.3 s against 122 s for the 2004 model and 128 s for F4—roughly fifty times faster—and it is the only method that penetrates the $n = 15$ block of the Catanzaro suite (21 of 36 attempted there). The largest instance solved by *any* method in the campaign has $n = 15$; no method solves anything at $n \geq 30$. The second observation concerns solver generations rather than formulations: under the same modern CPLEX, the 2004 formulation of Laporte et al. [29] equals F4 on the primary suite (91 solves each, near-identical mean times, the same wall at the $n = 15$, $m \approx 20$ block) and dominates it on the secondary suite: 667 against 455 solves, and twice as fast on the 454 instances both solve (13 against 26 s shifted geometric mean). Two of its runs crashed at the default memory allowance, were retried at 32 GB, and ended in ordinary timeouts; they are counted as failures. The moral is one of benchmarking hygiene: a comparison against an older formulation that does not state the solver generation measures the solver, not the formulation.

The Benders algorithm. The branch-and-Benders-cut solver has a pure *proof* problem, and the campaign quantifies it exactly: in all thirty time-limited runs of its best configuration, the incumbent held at the time limit already equalled the independently known optimum. The solver finds the answer, then spends the rest of the hour failing to certify it—typically ∼250,000 nodes and ∼75,000 optimality cuts with the dual bound still at its trivial root value, which on capacity-loose instances (where every pairwise bound w_{ij} vanishes) carries no information. The ablation is one-sided: the dual-cut

family is the only active axis (the configurations using LP-based dual cuts—BBC-LP in Table 4—solve 26 of 48 attempted secondary instances, the combinatorial-cut family none); triplet bounds are neutral; and the fractional Benders cuts recorded no additions across the campaign—which a post-campaign audit traced not to the cuts' strength but to a defect in the user-cut injection call (now corrected), so their effect on the dual bound is being re-measured on the corrected solver (Section 7). The method is bound-limited exactly in the sense of Section 5.3—and the correction that diagnosis dictates, the coverage row $\theta \geq |U|$ in the master, was implemented and re-run in full. The effect is decisive: on the primary suite the solve count rises from 1/9 to 43/73, and on the secondary one from 26/48 to 244/318 for the plain dual-cut arm and 274/374 for its triplet-bound variant, the deepest-reaching configuration; 122 of the dual-cut arm's 287 solves close at the root node, the median solve time is 0.05 s, and the ablation flattens—all eight configurations land within a narrow band, so the earlier dual-cut advantage was an artifact of the missing bound, not a property of the cut families. What does *not* change is equally informative: the corrected solver solves no instance outside the bound-tight class, and in all 104 of its remaining time-limited runs the incumbent again equals the independently known optimum. The Benders approach now solves exactly as far as its root bound reaches, and not one instance further—the residual failure is purely the loose-half bound, which is what the planned window inequalities target. Timing completes the picture: on the 43 primary instances that all four exact methods solve, the corrected Benders' shifted geometric-mean time is 1.0 s against 0.1 s for the flow model, 333 s for the 2004 model and 616 s for F4—within its reach, root closure outruns compact branch-and-bound by two orders of magnitude—and on the 244 commonly solved secondary instances it is on par with the 2004 model (12.9 against 10.7 s) and twice as fast as F4 (28.6 s), with median solve time 0.05 s and 90th percentile 197 s. The comparison is conditioned on the instances it solves, which are the bound-tight ones; that conditioning is the diagnosis itself.

The branch-and-price. On the validated range $n \leq 25$, PCF′ proves optimality on 94 of 319 attempted instances and PTF on 72 of 291; in the separate $n \geq 30$ extension run (96 attempts each) both solve nothing. The two prototypes fail differently, and the difference is exactly the relaxation. PCF′ solves where its root bound already equals the optimum (93 of its 94 solves) yet still needs search to *find* the integer solution: median 89 nodes, with only 16% of its solves finishing at a single node—bound tightness and integrality are different obstacles. PTF, whose relaxation is strictly stronger (Proposition 19), finishes 71 of its 72 solves at a single node: the added bound strength converts tree search into root closures, at the price of a pricing problem heavy enough that it solves fewer instances overall. The failure pattern is the two-regime diagnosis in numbers: of PCF′'s 225 failures on instances whose optimum is independently known, 73 lie on *bound-tight* instances—failures of speed alone, the direct target of the engineering agenda of Section 5.1—while 152 lie

on bound-loose instances that no acceleration can close without a stronger relaxation.

Bound tightness. Among the 806 instances solved by any method, the coverage bound $|U| - b$ equals the optimum on 387 (48.0%)—an upper-biased estimate, since unsolved instances skew loose; the 419 bound-loose solved instances are exactly the gap testbed used in the planned polyhedral work. The dichotomy of Section 3 is thus not a corner phenomenon: the standard benchmark suites split in half between the regime where the heuristic tends to be exact and exact methods close at the root, and the regime where both must work—which is where the position–transition relaxation and the planned polyhedral work aim.

5.3 The competitiveness of configuration branch-and-price

A guiding question for the exact methods is whether a configuration-based branch-and-price can be made fast for the SSP, and the structural results already constrain the answer. By Propositions 16 and 18 the configuration relaxation collapses to the coverage bound $|U| - b$, and by the gap theory of Section 3 this bound equals Z^* on a large part of the instance space. Where the root bound already equals the optimum branching is unnecessary; where it does not, a relaxation no stronger than $|U| - b$ forces a search tree that faster pricing or branching cannot avoid. The contrast with the Job Grouping Problem is telling: there the set-covering relaxation is strong enough that an analogous branch-and-price closes benchmark instances in a handful of nodes [39], whereas for the SSP the corresponding configuration relaxation does not improve on the trivial bound on most instances [36]. A configuration branch-and-price for the SSP is therefore *bound-limited* rather than implementation-limited. The position–transition formulation, the first in the family to exceed $|U| - b$ (Proposition 19), is the structural lever this diagnosis points to; but it pays off only where the gap opens, and broadening that improvement is part of the planned work (Section 7).

6 Conclusions

6.1 Summary

A single device—the equivalence between the SSP and a generalised travelling-salesman problem over tool configurations—has organised both a structural theory of the standard heuristic and a family of exact methods. On the structural side we proved that admitting arbitrary groupings makes the configuration framework exact (Theorem 1), so that the two-phase heuristic's gap is exactly the price of a minimum-cardinality grouping (Corollary 2); we bounded that gap from below and above (Corollary 1, Theorem 2), isolated zero-gap

classes valid for every capacity (Corollary 3), showed the gap can grow without bound (Proposition 5), and established a setup-cost spectrum whose two explicit thresholds carry the problem from the pure SSP to exact collapse onto grouping (Theorem 3, Lemma 4). Of the two natural conjectures, the 4/3 ratio at $b = 3$ is now proved whenever $K^* \leq 3$ (Proposition 10) and open only at $K^* \geq 4$; the gap bound $K^* - 2$ is proved on wide regimes, *refuted for the walk model* outside them by an explicit counterexample attaining our quantitative bounds with equality, and *reopened for the heuristic itself*, whose KTNS step beats the walk on the extremal instances (Propositions 7 and 9); the walk cost at $K^* = 3$ is given in closed form (Proposition 6), and $Z^* \leq 3$ forces gap at most one (Corollary 5). The circuit clutter of the grouping system computes K^* yet provably does not determine the gap, while blocking duality on the covering side yields the odd-ring non-idealness family for the JGP relaxation (Section 3.5). On the algorithmic side we gave a branch-and-Benders-cut solver whose subproblem is exact and polynomial, and position-indexed formulations with a fixed row set, among which the position–transition model is the first configuration relaxation to improve on the coverage bound. The methods are implemented; a first large-scale comparison is reported here, and the definitive campaign under the corrected protocol is running at the time of writing (Section 5.2), so the empirical claims of this report are stated as provisional.

6.2 Beyond the project horizon

The remaining work is set out in Section 7; three further directions fall outside it and are recorded here as lying beyond this project's horizon.

The position–transition device beyond the SSP. Nothing in the position–transition construction is specific to tool switching except the transition metric: position-transition columns with per-element consistency rows apply to any generalised travelling-salesman problem with overlapping clusters [42]. Whether the relaxation strength observed here persists in that generality is open, and an affirmative answer would carry the contribution beyond the SSP.

Approximation and online variants. The ratio theory of Section 3 invites a systematic worst-case study of the *other* SSP heuristics (constructive and TSP-based), none of which appears to carry a published guarantee; any such comparison must fix the switch-cost convention first. Separately, the caching equivalence of Remark 1 suggests an online SSP, in which jobs arrive over time, analysed competitively against paging machinery—with the offline loading oracle as a natural baseline.

Learning-augmented column generation and cut selection. Machine learning for the exact side is an active but crowded frontier—cut selection, ordering and removal [19],

and column and branching selection [6, 21, 35]—whose established role is to *choose* among cuts or columns one already knows how to generate rather than to strengthen them; the lifting itself stays combinatorial. The state of the art has moved from the policy-gradient cut selection of Tang et al. [48] to graph-neural-network state encoders trained by proximal policy optimisation and to hierarchical selection agents. A first prototype in this direction was built alongside this report—a policy-gradient agent that learns, from cut features, which cut most tightens the relaxation, validated to recover a sensible selection rule on a controlled cut family—with a graph-network encoder trained on the campaign's own cut traces as its state-of-the-art successor. On the pricing side the counterpart upgrade is not learning but the automatic dual stabilisation of Pessoa et al. [41], which would replace the static smoothing used here with an adaptive, self-combining scheme. Two observations, however, bound how much any of this can help the SSP in particular. First, the frontier of Section 5.3 is a *bound* gap, and cut selection cannot manufacture bound strength where no strong cut exists: on the instances that defeat the solver the incumbent already equals the optimum and only the dual bound lags, so a learned selector has nothing to select that would close them, and any useful signal must come from the structural imbalance between zero-gap and positive-gap instances rather than from search statistics. Second, the mature tooling for learned mixed-integer heuristics is built on the column-generation solver rather than the Benders master, which makes learned column selection for the position-indexed pricers [35]—whose coupled-pair pricing is the measured bottleneck—the more promising target, with cut selection for the Benders solver secondary. Each is a follow-up to a settled deterministic baseline, not a substitute for one.

7 Remaining work

Most of the programme this report set out to complete is done: the worst-case theory and the extremal case analysis (Sections 3.3–3.4), the structural characterisation (Section 3.5), the setup-cost spectrum (Section 3.6), and the full computational campaigns with their analysis (Section 5). Three items remain. They are independent of one another, and the first two are open research questions rather than pending tasks; no firm dates are attached, as an extended mid-summer break falls within the remaining period.

The exact worst case. Every known walk-extremal instance attains the bound of Proposition 7 with equality; for the heuristic itself the KTNS step closes those witnesses to gap 1 (Proposition 9), so the primary open question is now the exact worst case of the KTNS-evaluated heuristic—including whether gap $\leq K^* - 2$ holds for it—with the walk model's extremal theory as the proved scaffold. Alongside it stand the $K^* \geq 4$ regime (Proposition 8 is the current best bound) and the one remaining case of the 4/3 conjecture. First experiments are in: the bound gains two new attainment points at $R_{\text{opt}} = 1$, while

the $(5, 3)$ and $(6, 3)$ corners resist attainment under directed search—the question is now concentrated exactly there. In parallel, the exact formula of Proposition 6 turns grouping selection (Section 3.7) into a testable criterion.

The polyhedral programme. Two questions from Section 3.5 drive it: whether odd-ring-like structure is the *only* obstruction to integrality of the JGP covering relaxation, and which facets of the position–transition polytope explain where its bound exceeds $|U| - b$ (Proposition 19). A first scan is complete: the covering relaxation is non-integral on 24% of the $b = 3$ edge family, and non-integrality occurs off-ring (an explicit mixed-size witness has fractional cover 3.5 against $K^* = 4$)—so the obstruction landscape is richer than the odd rings alone, and the dataset for the characterisation question is prepared. The facet study on the small witness instances proceeds from there.

The corrected Benders master. The campaign showed that the Benders master lacked the elementary coverage bound every other method carries (Section 5.2). The correction is a single valid inequality, $\theta \geq |U|$, now implemented and verified. On bound-tight instances the corrected master closes at the root with no Benders cuts at all (on the 6-ring: zero cuts, against 443 before); on bound-loose instances it lifts the root to $|U|$ and search resumes from there. The full re-run with the corrected master is complete, and its outcome is reported in Section 5.2: the solve counts multiply, every remaining timeout still carries an optimal incumbent, and the configuration solves nothing outside the bound-tight class. The item that remains is therefore no longer the re-run but its successor: a bound that reaches into the loose half. The direction is fixed by an exact computation: solving the pairwise bound to optimality (a travelling-salesman path over the w_{ij}, exact for $n \leq 15$) lifts the root value on a quarter of the bound-loose instances—by 4.9 on average—yet reaches the optimum on under one per cent of them. Pairwise information is thus exhausted, and the next lever is its higher-order form: window inequalities over consecutive job sets S, charging $|\bigcup_{j \in S} T_j| - b$ when the jobs of S run contiguously, generalising the pairwise and triplet rows in one family.

Solver accelerations, implemented and awaiting evaluation. Four strengthenings of the Benders solver are implemented and checked against brute force on small instances; their campaign-scale evaluation is pending and is therefore not among the results above. First, the user-cut defect that suppressed every fractional Benders cut (Section 5.2) is corrected, so the cuts now enter the master—the question the re-run settles is whether they lift the dual bound above $|U| - b$ on bound-loose instances, which would qualify the bound-limited reading of Section 5.3. Second, a hybrid genetic search [33] supplies a strong root incumbent as a warm start and a seeded Benders cut. Third, conflict-graph clique inequalities contribute a valid root bound on instances whose grouping structure

dominates the coverage bound. Fourth, Papadakos-style Pareto-optimal lifting deepens the fractional cuts. Each is an independent option in the campaign harness, so the re-run measures the separate and combined effect of all four, together with the window inequalities above.

On the branch-and-price side, the four pricing accelerations of Section 4.4—heuristic pricing, multiple pricing, warm-start columns, and dual stabilisation—are likewise implemented as independent switches and verified to preserve exactness (IP $= Z^*$ against brute force on the ring and small random instances); their scale evaluation is part of the same pending campaign. A first learning-augmented cut-selection prototype (Section 6) is also in place but is deliberately gated on that campaign, since, for the reasons given there, it can help only if the run exposes bound-loose instances on which a learned selector would have something to act.

A Proofs

Proof of Theorem 1. Both inequalities are explicit constructions; by Proposition 1 we may assume the optimal schedule loads by KTNS, so every magazine is full, $|M_i| = b$.

($\leq$). Let $\mathcal{P} = \{G_1, \ldots, G_p\}$ be a feasible grouping with configurations $C_1, \ldots, C_p$, and let σ attain the minimum in Definition 5. Process the groups in the order $G_{\sigma(1)}, \ldots, G_{\sigma(p)}$, and within each group its jobs in any order, holding magazine $C_{\sigma(l)}$ throughout group $G_{\sigma(l)}$. This is feasible, since each $j \in G_{\sigma(l)}$ satisfies $T_j \subseteq \bigcup_{i \in G_{\sigma(l)}} T_i \subseteq C_{\sigma(l)}$ and $|C_{\sigma(l)}| = b$. The magazine is constant within a group and changes only at the $p-1$ group boundaries, so the free-initial switch cost is $\sum_{l=1}^{p-1} d(C_{\sigma(l)}, C_{\sigma(l+1)}) = \gamma(\mathcal{P})$. Hence $Z^* \leq \gamma(\mathcal{P})$, and minimising over $\mathcal{P}$ gives $Z^* \leq \min_{\mathcal{P}} \gamma(\mathcal{P})$.

($\geq$). Let $(\pi, \mathbf{M})$ be an optimal schedule with full magazines. Partition J into the maximal blocks of consecutive positions sharing one magazine; let these be $B_1, \ldots, B_r$ in order, with (size-b) magazines $D_1, \ldots, D_r$ and $D_l \neq D_{l+1}$. Each block is a group whose jobs run under D_l, so $\bigcup_{j \in B_l} T_j \subseteq D_l$ with $|D_l| = b$; the blocks thus form a feasible grouping $\mathcal{P}'$ with configurations $D_1, \ldots, D_r$. The identity ordering of these configurations is one admissible σ, so $\gamma(\mathcal{P}') \leq \sum_{l=1}^{r-1} d(D_l, D_{l+1})$, which is exactly the free-initial switch cost of $(\pi, \mathbf{M})$, namely Z^*. Therefore $\min_{\mathcal{P}} \gamma(\mathcal{P}) \leq \gamma(\mathcal{P}') \leq Z^*$. Combining the two inequalities gives equality. □

Proof of Proposition 5. The copies use pairwise disjoint tool sets, so $|U| = 6g$ and Proposition 3 gives $Z^* \geq 6g - 3$. For a matching schedule, process the copies one after another, each in the cyclic order of Example 1. Within a copy that loading makes 3 insertions to build the first magazine and 3 more across the copy; since consecutive copies are tool-disjoint, entering a new copy rebuilds its first magazine afresh (3 insertions). The cost is 6 insertions for the first copy and 6 for each of the others, i.e. $6g$ in the empty-start

count and $6g-3$ free-initial; hence $Z^* = 6g-3$.

For the heuristic, a minimum-cardinality grouping of R_g uses $K^* = 3g$ configurations, three per copy. In the ring, those three configurations are pairwise at distance 2, so any ordering of one copy's three contributes $2+2=4$; and any transition between configurations of different copies costs 3, the copies being tool-disjoint. Connecting g copies needs at least $g-1$ inter-copy transitions, so every ordering costs at least $4g+3(g-1) = 7g-3$, attained by keeping each copy's configurations consecutive. Thus $H = 7g-3$ and $H - Z^* = g$. □

Proof of Proposition 7: the walk-based claims. Suppose $Z^* = q$ and take an optimal schedule; its walk of distinct configurations $D_1, \ldots, D_p$ may be assumed, after excising configurations serving no job (the transition cost d obeys the triangle inequality $d(A,C) \le d(A,B) + d(B,C)$, and no covering walk costs less than q), to have every configuration serve a job. With zero re-insertions every insertion is first-time, so the $p-1$ transitions insert q tools in total and $p \le q+1$; assigning each job to a serving configuration exhibits a feasible p-grouping, so $p \ge K^*$; and if $p = K^*$ that grouping is minimum-cardinality of cost Z^*, forcing $H = Z^*$. In particular, $Z^* = q \le 2$ gives $p \le q+1 \le 3 = K^*$, so the gap vanishes—the claim used in part (i).

For (iv), let $q = 3$; if $p = 3$ the gap vanishes as above, so let $p = 4$. Each transition inserts exactly one fresh tool: $D_{l+1} \setminus D_l = \{x_{l+1}\}$ and $D_l \setminus D_{l+1} = \{e_{l+1}\}$; and zero re-insertion forces *persistence*: a tool of $D_i \cap D_j$ $(i < j)$ belongs to every intermediate D_l. Write U_l for the requirement of group G_l and suppose $|U_i \cup U_j| \le b$ for some $i < j$. Merging G_i and G_j, keeping the other two groups with their walk configurations, yields a minimum-cardinality grouping; it remains to choose its configuration $C \supseteq U_i \cup U_j$ and an ordering whose wrap term $|(\text{first} \cap \text{last}) \setminus \text{middle}|$ (Proposition 6) is at most one, giving $H \le q+1 = Z^*+1$. For $(1,2)$: take $C \subseteq D_1 \cup D_2$ (possible, the union has size $b+1$) and order (C, D_3, D_4); the wrap lies in $(D_1 \cap D_4 \setminus D_3) \cup (D_2 \cap D_4 \setminus D_3) \subseteq \varnothing \cup (D_2 \setminus D_3) = \{e_3\}$, using persistence. For $(3,4)$: order (D_1, D_2, C) with $C \subseteq D_3 \cup D_4$; the wrap lies in $D_1 \cap (D_3 \cup D_4) \setminus D_2 = \varnothing$ by persistence. For $(1,3)$, $(2,4)$, $(1,4)$: the orderings (C, D_2, D_4), (D_1, D_3, C), (C, D_2, D_3) bound the wrap by $\{x_3\}$, $\{e_3\}$, $\{x_3\}$ respectively, by the same two facts. For $(2,3)$: the set $W = U_2 \cup U_3 \cup (D_1 \cap D_4)$ lies in $D_2 \cup \{x_3\}$ (persistence places $D_1 \cap D_4$ inside $D_2 \cap D_3$), so $|W| \le b+1$ and a configuration $C \supseteq U_2 \cup U_3$ inside W omits at most one element of $D_1 \cap D_4$; the ordering (D_1, C, D_4) has wrap $|(D_1 \cap D_4) \setminus C| \le 1$. □

Proof of Proposition 20. Exactness. Take an optimal schedule and collapse equal consecutive magazines to a trajectory $D_1, \ldots, D_m$ $(m \le n)$ as in the proof of Theorem 1. Set $y^l_{D_l} = 1$ for $l = 1, \ldots, m$, leave positions $m+1, \ldots, n$ empty, and $w^k_t = \max(0, a^k_t - a^{k-1}_t)$. Then (P′) holds (= 1 on the prefix, = 0 after) and (G) holds (the prefix is contiguous), while (Cov), (W), (T) hold exactly as for PCF; the full-to-empty step $m \to m+1$ and all later steps add no cost, so the objective is $\sum_{l<m} d(D_l, D_{l+1}) = Z^*$. Hence the optimum is at most Z^*. Conversely, any integer-feasible PCF′ point has, by (G), its used positions

in a prefix $1, \ldots, p$, each holding one configuration D_k; by (W) the objective is at least $\sum_{k=2}^{p} d(D_{k-1}, D_k)$, and by (Cov) every job is served by some D_k, so assigning each job to a covering position yields a schedule of cost at most the objective. Hence the optimum is at least Z^*.

LP invariance. Every PCF-feasible point has $\sum_C y_C^k = 1$ for all k and so satisfies (P′) and (G); the PCF′ relaxation thus minimises over a superset and is no larger than PCF's. For the reverse, summing (T) over $t \in U$ and using $\sum_{t \in U} a_t^1 \le \sum_{t \in T} a_t^1 = b \sum_C y_C^1 \le b$ (the last bound now from (P′) rather than (P)) gives objective $\ge |U| - b$ at every PCF′ point, so the two relaxations coincide wherever PCF's value is $|U| - b$. The plain relaxation, dropping (T), is certified 0 by the same blended configuration used for PCF. □

References

[1] Najmaddin Akhundov and James Ostrowski. Exploiting symmetry for the job sequencing and tool switching problem. *European Journal of Operational Research*, 316(3):976–987, 2024.

[2] David L. Applegate, Robert E. Bixby, Vašek Chvátal, and William J. Cook. *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, 2006.

[3] Alper Atamtürk, George L. Nemhauser, and Martin W. P. Savelsbergh. Conflict graphs in solving integer programming problems. *European Journal of Operational Research*, 121(1):40–55, 2000.

[4] Cynthia Barnhart, Ellis L. Johnson, George L. Nemhauser, Martin W. P. Savelsbergh, and Pamela H. Vance. Branch-and-price: Column generation for solving huge integer programs. *Operations Research*, 46(3):316–329, 1998.

[5] László A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966.

[6] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: a methodological tour d'horizon. *European Journal of Operational Research*, 290(2):405–421, 2021.

[7] Alewyn P. Burger, C. G. Jacobs, Jan H. van Vuuren, and Stephan E. Visagie. Scheduling multi-colour print jobs with sequence-dependent setup times. *Journal of Scheduling*, 18(2):131–145, 2015.

[8] Dorothea Calmels. The job sequencing and tool switching problem: state-of-the-art literature review, classification, and trends. *International Journal of Production Research*, 57(15-16):5052–5068, 2018.

[9] Daniele Catanzaro, Luis Gouveia, and Martine Labbé. Improved integer linear programming formulations for the job sequencing and tool switching problem. *European Journal of Operational Research*, 244(3):766–777, 2015.

[10] Maria Chudnovsky, Neil Robertson, Paul Seymour, and Robin Thomas. The strong perfect graph theorem. *Annals of Mathematics*, 164(1):51–229, 2006.

[11] Gianni Codato and Matteo Fischetti. Combinatorial Benders' cuts for mixed-integer linear programming. *Operations Research*, 54(4):756–766, 2006.

[12] Rafael Colares and Annegret Wagler. Exact formulations for the job sequencing and tool switching problem. *Working Paper, Université Clermont Auvergne, LIMOS*, 2026.

[13] Rafael Colares, Mauricio de Souza, and Annegret Wagler. The tool switching problem. *Preprint, Université Clermont Auvergne, LIMOS*, 2026.

[14] Yves Crama and A. G. Oerlemans. A column generation approach to job grouping for flexible manufacturing systems. *European Journal of Operational Research*, 78(1): 58–80, 1994.

[15] Yves Crama, Antoon W. J. Kolen, Alwin G. Oerlemans, and Frits C. R. Spieksma. Minimizing the number of tool switches on a flexible machine. *International Journal of Flexible Manufacturing Systems*, 6(1):33–54, 1994.

[16] Yves Crama, Linda S. Moonen, Frits C. R. Spieksma, and Ellen Talloen. The tool switching problem revisited. *European Journal of Operational Research*, 182(2): 952–957, 2007.

[17] Antônio Augusto Chaves da Silva and Horacio Hideki Yanasse. A multicommodity flow model and exact methods for the job sequencing and tool switching problem. *Preprint (compiled 2024); see also Computers & Operations Research*, 2024. SSPMF multicommodity-flow formulation; LP lower bound $M - C$.

[18] Martin Desrochers and Gilbert Laporte. Improvements and extensions to the Miller–Tucker–Zemlin subtour elimination constraints. *Operations Research Letters*, 10(1): 27–36, 1991.

[19] Arnaud Deza and Elias B. Khalil. Machine learning for cutting planes in integer programming: A survey. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI)*, 2023.

[20] Matteo Fischetti, Juan José Salazar-González, and Paolo Toth. A branch-and-cut algorithm for the symmetric generalized traveling salesman problem. *Operations Research*, 45(3):378–394, 1997.

[21] Maxime Gasse, Didier Chételat, Nicola Ferroni, Laurent Charlin, and Andrea Lodi. Exact combinatorial optimization with graph convolutional neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

[22] Gianpaolo Ghiani, Antonio Grieco, and Emanuela Guerriero. Solving the job sequencing and tool switching problem as a nonlinear least cost hamiltonian cycle problem. *Networks*, 55(4):379–385, 2010.

[23] Olivier Goldschmidt, David Nehme, and Gang Yu. Note on the set-union knapsack problem. *Naval Research Logistics*, 41(6):833–842, 1994.

[24] Keld Helsgaun. Solving the equality generalized traveling salesman problem using the lin–kernighan–helsgaun algorithm. *Mathematical Programming Computation*, 7 (3):269–287, 2015.

[25] Alain Hertz, Gilbert Laporte, Michel Mittaz, and Kathryn E. Stecke. Heuristics for minimizing tool switches when scheduling part types on a flexible machine. *IIE Transactions*, 30(8):689–694, 1998.

[26] John N. Hooker and Greger Ottosson. Logic-based benders decomposition. *Mathematical Programming*, 96(1):33–60, 2003.

[27] Roel Jans and Jacques Desrosiers. Efficient symmetry breaking formulations for the job grouping problem. *Computers & Operations Research*, 40(4):1132–1142, 2013.

[28] Layth Kashou, André Rossi, Evgeny Gurevsky, and Rui S. Shibasaki. Benders decomposition for robust balancing of production lines. M2 internship report, Université Paris-Saclay (LAMSADE, Université Paris-Dauphine–PSL), 2025.

[29] Gilbert Laporte, Juan José Salazar-González, and Frédéric Semet. Exact algorithms for the job sequencing and tool switching problem. *IIE Transactions*, 36(1):37–45, 2004.

[30] Emma Legrand, Vianney Coppé, Daniele Catanzaro, and Pierre Schaus. A dynamic programming approach for the job sequencing and tool switching problem. In *Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR 2025)*, volume 15763 of *Lecture Notes in Computer Science*, pages 70–85. Springer, 2025.

[31] Marco E. Lübbecke and Jacques Desrosiers. Selected topics in column generation. *Operations Research*, 53(6):1007–1023, 2005.

[32] Thomas L. Magnanti and Richard T. Wong. Accelerating Benders decomposition: Algorithmic enhancement and model selection criteria. *Operations Research*, 29(3): 464–484, 1981.

[33] Jordana Mecler, Anand Subramanian, and Thibaut Vidal. A simple and effective hybrid genetic search for the job sequencing and tool switching problem. *Computers & Operations Research*, 127:105153, 2021.

[34] Clair E. Miller, Albert W. Tucker, and Richard A. Zemlin. Integer programming formulation of traveling salesman problems. *Journal of the ACM*, 7(4):326–329, 1960.

[35] Mouad Morabit, Guy Desaulniers, and Andrea Lodi. Machine-learning-based column selection for column generation. *Transportation Science*, 55(4):815–831, 2021.

[36] Bernardo Bastos Pereira Moreira. A new combinatorial approach to solve the job sequencing and tool switching problem. Master's thesis, Universidade Federal de Minas Gerais (UFMG), Belo Horizonte, Brazil, 2026.

[37] İbrahim Muter, Ş. İlker Birbil, and Kerem Bülbül. Simultaneous column-and-row generation for large-scale linear programs with column-dependent-rows. *Mathematical Programming*, 142(1–2):47–82, 2013.

[38] Charles E. Noon and James C. Bean. An efficient transformation of the generalized traveling salesman problem. *INFOR: Information Systems and Operational Research*, 31(1):39–44, 1993.

[39] Felipe Otiai. Flexible manufacturing systems: Job grouping and tool management. Master's thesis, Federal University of Minas Gerais, 2026.

[40] Nikolaos Papadakos. Practical enhancements to the Magnanti–Wong method. *Operations Research Letters*, 36(4):444–449, 2008.

[41] Artur Pessoa, Ruslan Sadykov, Eduardo Uchoa, and François Vanderbeck. Automation and combination of linear-programming based stabilization techniques in column generation. *INFORMS Journal on Computing*, 30(2):339–360, 2018.

[42] Petrică C. Pop, Ovidiu Cosma, Cosmin Sabo, and Corina Pop Sitar. The generalized traveling salesman problem: A comprehensive review. *European Journal of Operational Research*, 314(3):819–835, 2024.

[43] Caroline Privault and Gerd Finke. Modelling a tool switching problem on a single nc-machine. *Journal of Intelligent Manufacturing*, 6(2):87–94, 1995.

[44] Caroline Privault and Gerd Finke. K-server problems with bulk requests: An application to tool switching in manufacturing. *Annals of Operations Research*, 96 (1/4):255–269, 2000.

[45] Ragheb Rahmaniani, Teodor Gabriel Crainic, Michel Gendreau, and Walter Rei. The Benders decomposition algorithm: A literature review. *European Journal of Operational Research*, 259(3):801–817, 2017.

[46] D. M. Ryan and B. A. Foster. An integer programming approach to scheduling. *Computer Scheduling of Public Transport*, pages 269–280, 1981.

[47] Christopher S. Tang and Eric V. Denardo. Models arising from a flexible manufacturing machine, part i: Minimization of the number of tool switches. *Operations Research*, 36(5):767–777, 1988.

[48] Yunhao Tang, Shipra Agrawal, and Yuri Faenza. Reinforcement learning for integer programming: Learning to cut. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *PMLR*, pages 9367–9376, 2020.

[49] François Vanderbeck and Laurence A. Wolsey. Reformulation and decomposition of integer programs. In Michael Jünger et al., editors, *50 Years of Integer Programming 1958–2008*, pages 431–502. Springer, Berlin, Heidelberg, 2010.

[50] Horacio Hideki Yanasse, Ricardo de Carvalho Miranda Rodrigues, and Edson Luiz França Senne. An enumerative algorithm based on partial ordering for the minimization of tool switches problem. *Gestão & Produção*, 16(3):303–314, 2009.